

11 Module 11: Mass Storage Management

11.1 Course Wrap-up Video

11.1.1 Course Completion and Congratulations

- The instructor congratulates students on successfully completing the **Operating Systems** course.
- Acknowledges the **diligent engagement** with course materials and the progress made throughout the term.
- Emphasizes the importance of revisiting **learning objectives** to reinforce comprehension before concluding the course.

11.1.2 Recap of Core Learning Objectives

11.1.2.1 1. Fundamental Role of Operating Systems

- **Definition and Purpose:**
 - Operating systems (OS) act as an **intermediary** between **computer hardware** and **software applications**.
 - Primary function: **Manage hardware resources** and **provide a stable, efficient environment** for software execution.
- **Key Responsibilities:**
 - **Abstraction:** Hides complex hardware details from applications via system calls and APIs.
 - **Resource Allocation:** Distributes CPU time, memory, and I/O devices among competing processes.
 - **Protection and Security:** Enforces access controls to prevent unauthorized or malicious operations.

11.1.2.2 2. Operating System Structure and Components

- **Architectural Overview:**
 - Studied the **layered structure** of modern OS (e.g., kernel, system libraries, user interface).
 - Explored **monolithic**, **microkernel**, and **hybrid** designs.
- **Core Services:**
 1. **System Initialization (Booting):**
 - Process of loading the OS into memory and preparing the system for operation.
 - Involves **BIOS/UEFI**, **bootloaders**, and **kernel initialization**.
 2. **Device Management:**
 - **Device drivers** facilitate communication between hardware (e.g., printers, disks) and the OS.
 - **I/O scheduling** optimizes data transfer efficiency (e.g., disk arm movement in HDDs).
 3. **User Interface:**
 - **Command-line interfaces (CLI)** vs. **graphical user interfaces (GUI)**.
 - Trade-offs between **resource usage** and **usability**.

11.1.2.3 3. Process and Thread Management

11.1.2.3.1 3.1 Process Lifecycle and Creation

- **Definition:** A **process** is an instance of a program in execution, with its own **memory space**, **registers**, and **state**.
- **States of a Process:**
 - New -> Ready -> Running -> Waiting -> Terminated.
- **Process Creation Mechanisms:**
 - **Fork()** (Unix/Linux): Creates a child process as a copy of the parent.
 - **Exec()**: Replaces the current process image with a new program.

- **Thread Creation:** Lighter-weight than processes; shares memory space with the parent process.

11.1.2.3.2 3.2 Process Scheduling

- **Goals:**
 - Maximize **CPU utilization**.
 - Ensure **fairness** and **responsiveness**.
 - Minimize **turnaround time** and **waiting time**.
- **Scheduling Algorithms:**
 - **First-Come, First-Served (FCFS):** Non-preemptive; may lead to the **convoy effect**.
 - **Shortest Job First (SJF):** Optimal for turnaround time but requires knowledge of burst times.
 - **Priority Scheduling:** Assigns priority levels; risk of **starvation** for low-priority processes.
 - **Round Robin (RR):** Preemptive; uses a **time quantum** to ensure fairness.
 - **Multilevel Queue:** Separates processes into queues (e.g., foreground vs. background).
 - **Multilevel Feedback Queue:** Allows processes to move between queues dynamically.

11.1.2.3.3 3.3 Process Synchronization

- **Critical Section Problem:**
 - Ensures **mutual exclusion**, **progress**, and **bounded waiting** when multiple processes access shared resources.
- **Solutions:**
 - **Locks (Mutexes):** Binary mechanism to protect critical sections.
 - **Semaphores:** Integer-based synchronization (counting vs. binary semaphores).
 - **Monitors:** High-level constructs with **condition variables** (e.g., in Java).
 - **Deadlock-Free Designs:** E.g., **Peterson's algorithm** for two processes.
- **Classical Synchronization Problems:**
 - **Producer-Consumer Problem:** Buffer management with bounded capacity.
 - **Dining Philosophers Problem:** Resource allocation with circular wait.
 - **Readers-Writers Problem:** Balancing read/write access to shared data.

11.1.2.3.4 3.4 Deadlocks: Prevention, Avoidance, and Recovery

- **Definition:** A state where **two or more processes** are **blocked forever**, each waiting for a resource held by another.
- **Necessary Conditions (Coffman Conditions):**
 1. **Mutual Exclusion:** Only one process can use a resource at a time.
 2. **Hold and Wait:** A process holds a resource while waiting for another.
 3. **No Preemption:** Resources cannot be forcibly taken from a process.
 4. **Circular Wait:** A cycle exists where each process waits for a resource held by another.
- **Strategies:**
 - **Prevention:** Break one of the four conditions (e.g., **resource ordering**, **timeout-based preemption**).
 - **Avoidance:** Use **Banker's Algorithm** to dynamically check for safe states.
 - **Detection and Recovery:**
 - * **Detection:** Construct a **wait-for graph** to identify cycles.
 - * **Recovery:**
 - **Process Termination:** Kill one or more processes in the deadlock.
 - **Resource Preemption:** Roll back a process and reallocate resources.

11.1.2.4 4. Memory Management

11.1.2.4.1 4.1 Partitioning and Allocation

- **Contiguous Memory Allocation:**
 - **Fixed Partitioning:** Predefined memory blocks; suffers from **internal fragmentation**.
 - **Dynamic Partitioning:** Allocates memory as needed; risks **external fragmentation**.
 - **Compaction:** Relocates processes to consolidate free memory (requires dynamic relocation).
- **Non-Contiguous Memory Allocation:**
 - **Paging:**
 - * Divides memory into fixed-size **frames** and processes into **pages**.
 - * Uses a **page table** for address translation.
 - * **Advantages:** No external fragmentation; efficient memory usage.
 - * **Overhead:** Page table storage and **TLB (Translation Lookaside Buffer)** management.
 - **Segmentation:**
 - * Divides memory into variable-sized **segments** (e.g., code, data, stack).
 - * **Advantages:** Logical organization; easier sharing/protection.
 - * **Disadvantages:** External fragmentation; complex management.

11.1.2.4.2 4.2 Virtual Memory

- **Definition:** A technique that **abstracts physical memory** to provide each process with an **illusion of a large, contiguous address space**.
- **Key Mechanisms:**
 - **Demand Paging:** Loads pages into memory **only when needed** (reduces I/O overhead).
 - **Page Replacement Algorithms:**
 - * **FIFO (First-In-First-Out):** Simple but may cause **Belady's anomaly**.
 - * **LRU (Least Recently Used):** Optimal in theory but expensive to implement.
 - * **Clock Algorithm:** Approximates LRU with lower overhead.
 - **Thrashing:** Excessive page faults due to **insufficient frames**; mitigated via:
 - * **Working Set Model:** Keeps a process's most frequently used pages in memory.
 - * **Page Fault Frequency (PFF):** Adjusts frame allocation dynamically.

11.1.2.5 5. Secondary Storage Management

11.1.2.5.1 5.1 Storage Device Overview

- **Hard Disk Drives (HDDs):**
 - **Mechanical components:** Platters, read/write heads, spindle.
 - **Access Time:** Sum of **seek time**, **rotational latency**, and **transfer time**.
 - **Optimization:** **Disk scheduling algorithms** (e.g., **SCAN**, **C-SCAN**, **SSTF**).
- **Solid State Drives (SSDs):**
 - **No moving parts;** uses **flash memory** (NAND gates).
 - **Advantages:** Faster access, lower latency, shock resistance.
 - **Disadvantages:** Limited **write endurance**; requires **wear leveling** and **garbage collection**.

11.1.2.5.2 5.2 File System Organization

- **File System Structure:**
 - **Logical Storage:** Files and directories (hierarchical organization).
 - **Physical Storage:** Blocks/sector allocation (e.g., **FAT**, **NTFS**, **ext4**).
- **Allocation Methods:**
 - **Contiguous Allocation:** Simple but leads to fragmentation.

- **Linked Allocation:** Uses pointers; no external fragmentation but slow random access.
- **Indexed Allocation:** Stores block pointers in an **index block**; supports direct access.
- **Free Space Management:**
 - **Bit Vector:** Each bit represents a block (1 = free, 0 = allocated).
 - **Linked List:** Chains free blocks; simple but inefficient for large searches.
 - **Grouping:** Stores addresses of multiple free blocks in the first free block.

11.1.2.5.3 5.3 Optimization Techniques

- **Disk Defragmentation:** Reorganizes fragmented data to improve access speed (relevant for HDDs).
- **Caching:** Uses **buffer cache** to reduce disk I/O (e.g., **page cache** in Linux).
- **RAID (Redundant Array of Independent Disks):**
 - **RAID 0:** Striping for performance (no redundancy).
 - **RAID 1:** Mirroring for redundancy.
 - **RAID 5:** Striping with distributed parity (balance of performance and fault tolerance).

11.1.3 Course Summary and Future Applications

- **Comprehensive Understanding Achieved:**
 - Students now grasp **core OS principles**, from **low-level hardware interactions** to **high-level resource management**.
 - Skills acquired are **directly applicable** to:
 - * **Advanced studies** in computer science (e.g., distributed systems, real-time OS).
 - * **Professional settings** (e.g., system administration, software development, cybersecurity).
- **Final Encouragement:**
 - The instructor reiterates **congratulations** and wishes students **continued success** in their academic and professional pursuits.

11.2 C-SCAN Disk Scheduling Algorithm

11.2.1 1. Introduction to C-SCAN

- **Definition:** C-SCAN (Circular SCAN) is a **variant of the SCAN disk scheduling algorithm** designed to provide **more uniform wait times** for disk requests.
- **Core Concept:** Treats the disk as a **circular list**, unlike the traditional SCAN algorithm.
- **Primary Goal:** To **minimize variance in wait times** across different disk requests, improving fairness in request servicing.

11.2.2 2. How C-SCAN Works

11.2.2.1 2.1 Basic Operation

1. The disk head **moves from one end of the disk to the other**, servicing requests **only in one direction** (e.g., from the innermost to the outermost cylinder).
2. Upon reaching the **end of the disk**, the head **immediately jumps back to the beginning (opposite end) without servicing any requests** during the return trip.
3. The process **repeats**, with the head moving in the **same direction** again, servicing new and pending requests.

11.2.2.2 2.2 Key Difference from SCAN

Feature	SCAN (Elevator Algorithm)	C-SCAN (Circular SCAN)
Directionality	Services requests in both directions (back and forth).	Services requests in only one direction , then resets.
Return Trip	Services requests while moving backward.	Does not service requests on the return trip.
Wait Time Uniformity	Higher variance in wait times (requests near the ends may wait longer).	More uniform wait times due to unidirectional servicing.

11.2.3 3. Step-by-Step Example of C-SCAN

11.2.3.1 3.1 Scenario Setup

- **Disk Configuration:**
 - Total cylinders: **200 (numbered 0-199)**.
 - Initial head position: **Cylinder 53**.
- **Request Queue (8 requests):**
 - 65, 67, 98, 122, 124, 183, 14, 37.

11.2.3.2 3.2 Execution Steps

1. **First Pass (Moving Toward Higher Cylinders):**
 - The head moves from **53 -> 65** (services 65).
 - **65 -> 67** (services 67).
 - **67 -> 98** (services 98).
 - **98 -> 122** (services 122).
 - **122 -> 124** (services 124).
 - **124 -> 183** (services 183).
 - **183 -> 199** (reaches end, **no request serviced here**).
2. **Wrap-Around (Jump Back to Start):**
 - The head **jumps from 199 -> 0** (no servicing during this movement).
3. **Second Pass (Moving Toward Higher Cylinders Again):**
 - From **0 -> 14** (services 14).
 - **14 -> 37** (services 37).
 - The process **repeats** for any remaining requests.

11.2.3.3 3.3 Total Head Movement Calculation

Movement Segment	Cylinders Moved	Cumulative Movement
53 -> 65	12	12
65 -> 67	2	14
67 -> 98	31	45
98 -> 122	24	69
122 -> 124	2	71
124 -> 183	59	130
183 -> 199	16	146
199 -> 0 (Wrap-Around)	199	345
0 -> 14	14	359
14 -> 37	23	382

- **Total Head Movement: 382 cylinders.**

11.2.4 4. Advantages of C-SCAN

1. **Uniform Wait Times:**
 - Since requests are serviced in **only one direction**, the algorithm **reduces starvation** for requests near the ends of the disk.
 - Avoids the “**elevator effect**” (where requests at extremes wait longer in SCAN).
2. **Predictable Performance:**
 - The **deterministic movement pattern** makes it easier to **estimate response times**, beneficial in **real-time systems**.
3. **Simpler Implementation:**
 - Unidirectional movement **reduces complexity** in tracking head direction changes (unlike SCAN, which requires bidirectional logic).
4. **Efficiency in High-Load Scenarios:**
 - Performs well in systems with a **large number of requests**, as it **minimizes seek time variance**.

11.2.5 5. Disadvantages of C-SCAN

1. **Overhead from Wrap-Around:**
 - The **jump from the end to the start** (e.g., 199 -> 0) introduces **significant head movement** without servicing any requests, leading to **inefficiency in some cases**.
2. **Inefficiency with Sparse or Clustered Requests:**
 - If requests are **concentrated at one end** of the disk, C-SCAN may **waste time** moving across empty regions.
 - Example: If most requests are near **Cylinder 199**, the head still **scans all the way to 0** before returning, increasing **unnecessary seek time**.
3. **Potential for Longer Average Seek Times:**
 - In some workloads, **SCAN or LOOK** (optimized SCAN) may outperform C-SCAN due to **bidirectional servicing**.

11.2.6 6. Comparison with Other Disk Scheduling Algorithms

Algorithm	Directionality	Wait Time Uniformity	Overhead	Best Use Case
FCFS	Arbitrary	Poor (starvation possible)	Low	Simple systems
SSTF	Shortest seek first	Moderate (starvation risk)	Low	Low-load systems
SCAN	Bidirectional	Moderate (ends suffer)	Moderate	General-purpose
C-SCAN	Unidirectional	High (uniform)	High (wrap-around)	Fairness-critical systems
LOOK	Bidirectional (optimized)	Better than SCAN	Low	High-performance systems

11.2.7 7. When to Use C-SCAN?

- **Applications Requiring Fairness:**
 - Systems where **uniform response times** are critical (e.g., **real-time databases, multimedia streaming**).
- **High Request Density:**
 - Works well when requests are **evenly distributed** across the disk.

- **Predictable Workloads:**
 - Useful in **embedded systems** where **deterministic behavior** is needed.

11.2.8 8. When to Avoid C-SCAN?

- **Sparse or Clustered Requests:**
 - If requests are **heavily concentrated** in one region, **SCAN or LOOK** may be more efficient.
- **Low-Latency Requirements:**
 - The **wrap-around overhead** can introduce **unnecessary delays** in time-sensitive applications.
- **Energy-Constrained Systems:**
 - The **long seek during wrap-around** consumes more power, which may be undesirable in **mobile or battery-operated devices**.

11.2.9 9. Summary of Key Points

- C-SCAN is a **unidirectional** disk scheduling algorithm that **treats the disk as a circular list**.
- It **services requests in one direction**, then **jumps back to the start** without servicing on the return.
- **Advantages:**
 - More **uniform wait times**.
 - **Predictable performance**.
 - **Simpler implementation** than SCAN.
- **Disadvantages:**
 - **Overhead from wrap-around**.
 - **Inefficient for sparse/clustered requests**.
- **Best for:** Systems requiring **fairness and predictable response times**.
- **Avoid when:** Requests are **unevenly distributed** or **low latency is critical**.

11.2.10 10. Conclusion

C-SCAN is a **valuable disk scheduling algorithm** for scenarios where **consistent performance** and **fairness** are prioritized over raw speed. While it introduces some **overhead due to wrap-around**, its **uniform wait times** make it suitable for **real-time and high-load systems**. However, in cases where requests are **sparse or clustered**, alternative algorithms like **SCAN or LOOK** may be more efficient.

11.3 Disk Attachment

11.3.1 Introduction to Disk Attachment Methods

- Disk storage is accessed by computers through two primary methods:
 1. **Host Attached Storage:** Accessed via local I/O ports (common in smaller systems).
 2. **Network Attached Storage (NAS):** Accessed via a remote host in a distributed file system.

11.3.2 Host Attached Storage

11.3.2.1 Definition

- Storage accessed through **local I/O ports** using various technologies.
- Commonly used in **smaller computer systems** (e.g., desktops, workstations).

11.3.2.2 Technologies and Protocols

1. **IDE (Integrated Drive Electronics) / ATA (Advanced Technology Attachment)**

- Traditional **parallel bus architecture** for desktop PCs.
 - Supports a **maximum of two drives per I/O bus**.
 - **Limitation:** Parallel signaling leads to cable clutter and limited scalability.
2. **SATA (Serial Advanced Technology Attachment)**
- **Serial successor** to IDE/ATA, offering **simplified cabling** and improved performance.
 - Uses **point-to-point connections** (one drive per cable).
 - **Advantages:**
 - Thinner, longer cables (up to 1 meter).
 - Hot-swappable drives (in some implementations).
 - Higher data transfer rates (e.g., SATA III at 6 Gbps).
3. **Fibre Channel (FC)**
- **High-speed serial architecture** used in **high-end workstations and servers**.
 - Operates over **optical fiber or copper cable**.
 - Two variants:
 - **Switched Fabric:**
 - * Uses a **24-bit address space**.
 - * Forms the **basis for Storage Area Networks (SANs)**.
 - * Enables **scalable, high-performance storage networks**.
 - **Arbitrated Loop (FC-AL):**
 - * Supports **up to 126 devices** in a loop topology.
 - * Lower cost but less scalable than switched fabric.

11.3.2.3 Supported Storage Devices

- **Hard Disk Drives (HDDs)**
- **RAID (Redundant Array of Independent Disks) arrays**
- **Optical drives (CD, DVD)**
- **Tape drives**

11.3.2.4 I/O Commands

- Data transfers are initiated via **I/O commands** for:
 - **Read/write operations** on **logical data blocks**.
 - Targeted to **specifically identified storage units** (e.g., bus ID, target logical unit).

11.3.3 Network Attached Storage (NAS)

11.3.3.1 Definition

- **Special-purpose storage systems** accessed **remotely over a data network**.
- Clients interact with NAS using **Remote Procedure Calls (RPC)**.

11.3.3.2 Protocols

1. **NFS (Network File System)**
 - Used in **UNIX/Linux systems**.
 - Enables **file-level access** over a network.
2. **CIFS (Common Internet File System)**
 - Used in **Windows environments**.
 - Provides **file and printer sharing** over a network.

11.3.3.3 Network Transport

- RPCs are carried over **TCP or UDP** via an **IP network**.

11.3.3.4 Advantages of NAS

- **Convenient sharing**: All computers on a **Local Area Network (LAN)** can access a **shared storage pool**.
- **Familiar interface**: Mimics **local storage access** (e.g., file paths, permissions).
- **Disadvantages**:
 - **Lower performance** compared to direct-attached storage (due to network overhead).
 - **Bandwidth consumption** on the data network.

11.3.4 iSCSI (Internet Small Computer Systems Interface)

11.3.4.1 Definition

- A **network-attached storage protocol** that **encapsulates SCSI commands** over an **IP network**.
- Enables **block-level storage access** using existing **Ethernet infrastructure**.

11.3.4.2 Key Features

- **Replaces traditional SCSI cables** with **network connections** (e.g., Ethernet, Wi-Fi).
- **Leverages IP networks** for storage communication.
- **Advantages**:
 - **Cost-effective**: Uses existing network hardware (switches, routers).
 - **Scalable**: Easy to expand storage capacity.
 - **Centralized management**: Simplifies storage administration.
 - **Location flexibility**: Access storage from **any network-connected location**.

11.3.4.3 Use Cases

- **Enterprise storage** (e.g., data centers).
- **Virtualization environments** (e.g., VMware, Hyper-V).
- **Disaster recovery** (remote storage replication).

11.3.5 Storage Area Network (SAN)

11.3.5.1 Motivation: Drawbacks of NAS

1. **Bandwidth consumption**: Storage I/O operations **compete with general data traffic** on the network.
2. **Latency issues**: **Increased communication delays** in large client-server installations.

11.3.5.2 Definition

- A **private network** dedicated to **storage traffic**.
- Uses **storage protocols** (e.g., Fibre Channel, iSCSI) instead of **general network protocols** (e.g., TCP/IP).

11.3.5.3 Architecture

- Connects **servers and storage units** via a **high-speed network**.
- **SAN switches** control access between hosts and storage devices.
- **Dynamic allocation**: Storage can be **reassigned to different hosts** as needed.

11.3.5.4 Interconnect Technologies

1. Fibre Channel (FC)

- **Most common SAN interconnect.**
- **High performance** (up to 128 Gbps in FC-NVMe).
- Supports **both optical and copper connections.**

2. iSCSI

- **Growing in popularity** due to **simplicity and cost-effectiveness.**
- Runs over **standard Ethernet networks.**

3. InfiniBand

- **Special-purpose bus architecture** for high-speed interconnections.
- Provides **hardware/software support** for:
 - **Low-latency communication.**
 - **High throughput** (up to 400 Gbps).
- Used in **supercomputing and data centers.**

11.3.5.5 Advantages of SAN

- **Flexibility:**
 - Multiple hosts and storage arrays can **attach to the same SAN.**
 - Storage can be **dynamically allocated/reallocated.**
- **Performance:**
 - **Dedicated network** minimizes latency and bandwidth contention.
 - **High-speed protocols** (e.g., Fibre Channel) optimize storage access.
- **Scalability:**
 - Easy to **add more storage or servers** without disrupting operations.
- **Reliability:**
 - Supports **redundant paths** (multipath I/O) for fault tolerance.

11.3.6 Comparison of Disk Attachment Methods

Feature	Host Attached Storage	Network Attached Storage (NAS)	Storage Area Network (SAN)
Access Method	Local I/O ports (IDE, SATA, FC)	Remote via network (NFS, CIFS)	Dedicated storage network (FC, iSCSI)
Performance	High (direct access)	Moderate (network-dependent)	Very high (optimized for storage)
Scalability	Limited (e.g., 2 drives per IDE bus)	High (shared over LAN)	Very high (supports many devices)
Complexity	Low	Moderate	High (requires specialized hardware)
Use Case	Desktops, workstations	Small/medium businesses	Enterprises, data centers
Protocol Examples	ATA, SATA, Fibre Channel	NFS, CIFS	FC, iSCSI, InfiniBand
Bandwidth Competition	None (dedicated I/O)	Shares LAN bandwidth	Dedicated storage network

11.3.7 Summary of Key Concepts

1. Disk attachment methods:

- **Host Attached Storage:** Local I/O (IDE, SATA, Fibre Channel).
 - **Network Attached Storage (NAS):** Remote access via NFS/CIFS over IP.
 - **Storage Area Network (SAN):** Dedicated high-speed storage network.
2. **Protocols and Technologies:**
- **IDE/ATA:** Legacy parallel bus (2 drives max).
 - **SATA:** Serial successor to ATA (simplified cabling).
 - **Fibre Channel:** High-speed serial (used in SANs).
 - **iSCSI:** SCSI over IP (cost-effective, scalable).
 - **InfiniBand:** High-performance interconnect for servers/storage.
3. **NAS vs. SAN:**
- **NAS: File-level access**, shares LAN bandwidth, easier to manage.
 - **SAN: Block-level access**, dedicated network, higher performance.
4. **Trends:**
- **iSCSI** is gaining traction due to **cost savings** and **Ethernet compatibility**.
 - **SANs** remain critical for **high-performance, large-scale storage** (e.g., data centers).

11.4 Disk Structure

11.4.1 1. Introduction to Modern Magnetic Disk Drives

- Modern magnetic disk drives store data as a **large one-dimensional array of logical blocks**.
- These **logical blocks** are the smallest units of data transfer, typically **512 bytes** in size.
 - Some disks can be formatted with different block sizes (e.g., **1,024 bytes**).
- Logical blocks are **sequentially mapped** onto the **sectors** of the disk.

11.4.2 2. Mapping Logical Blocks to Physical Sectors

11.4.2.1 2.1 Sequential Mapping Process

- The mapping begins with **sector 0**, which is the **first sector of the first track on the outermost cylinder**.
- The sequence proceeds:
 1. Through the **remaining sectors of the first track**.
 2. Through the **rest of the tracks in the same cylinder**.
 3. Moves **inward** from the **outermost cylinder** to the **innermost cylinder**.

11.4.2.2 2.2 Challenges in Address Translation

- **Theoretical possibility:** A **logical block number (LBN)** can be converted into a **physical address** consisting of:
 - **Cylinder number**
 - **Track number**
 - **Sector number**
- **Practical challenges:**
 1. **Defective sectors:** Some sectors may be faulty and are **remapped to spare sectors**.
 2. **Variable sectors per track:** The number of sectors per track is **not constant** on some drives, complicating direct mapping.

11.4.3 3. Disk Rotation and Data Transfer Methods

11.4.3.1 3.1 Constant Linear Velocity (CLV)

- **Definition:** A method where the **disk spin speed adjusts** to maintain a **constant data transfer rate**.

- **Characteristics:**
 - **Uniform bit density per track** (same number of bits per unit length).
 - **Outer tracks** (farther from the center) are **longer** and thus **hold more sectors**.
 - **Inner tracks** (closer to the center) have **fewer sectors**.
 - To maintain a **consistent data rate**, the **rotation speed increases** as the read/write head moves **inward**.
- **Applications:**
 - Used in **CD-ROM** and **DVD-ROM** drives.

11.4.3.2 3.2 Constant Angular Velocity (CAV)

- **Definition:** A method where the **disk rotates at a constant speed**.
- **Characteristics:**
 - **Bit density decreases** from **inner tracks** to **outer tracks** to maintain a **constant data rate**.
 - **Outer tracks** have **lower bit density** but **more sectors** due to longer circumference.
 - **Inner tracks** have **higher bit density** but **fewer sectors**.
- **Applications:**
 - Commonly used in **hard disk drives (HDDs)**.

11.4.4 4. Technological Advancements in Disk Structure

11.4.4.1 4.1 Zoned Bit Recording (ZBR)

- **Definition:** A technique where **tracks are grouped into zones** (e.g., Zone 0, Zone 1, etc.).
- **Key Features:**
 - **Tracks within the same zone** have a **fixed number of sectors**.
 - **Outer zones** (farther from the center) can have **up to 40% more sectors** than **inner zones**.
 - Improves **storage efficiency** by optimizing sector distribution based on track length.

11.4.4.2 4.2 Increase in Cylinder Count

- Modern disks have seen a **significant increase in the number of cylinders**.
- Large disks now contain **tens of thousands of cylinders**, enhancing **storage capacity**.

11.4.5 5. Summary of Key Concepts

1. **Logical Blocks & Physical Sectors:**
 - Data is stored as **logical blocks** (typically 512 bytes), mapped to **physical sectors**.
 - Mapping follows a **sequential order** from the outermost cylinder inward.
2. **Address Translation Challenges:**
 - **Defective sectors** require remapping to spares.
 - **Variable sectors per track** complicate direct LBN-to-physical-address conversion.
3. **Rotation Methods:**
 - **CLV (Constant Linear Velocity):** Adjusts spin speed for uniform bit density (used in optical drives).
 - **CAV (Constant Angular Velocity):** Fixed spin speed with varying bit density (used in HDDs).
4. **Technological Improvements:**
 - **Zoned Bit Recording (ZBR):** Increases sector count in outer zones.
 - **Higher cylinder counts:** Modern disks support **tens of thousands of cylinders**.

11.5 FCFS Disk Scheduling Algorithm

11.5.1 1. Introduction to FCFS Disk Scheduling

- **Definition:**
 - **FCFS (First-Come, First-Served)** is the simplest disk scheduling algorithm.
 - It processes disk I/O requests in the **exact order** they arrive in the disk request queue.
 - This method is **intrinsically fair** because it treats all requests equally without prioritization.
- **Context:**
 - Part of **Mass Storage Management** in operating systems.
 - Focuses on optimizing disk head movement to improve I/O performance.

11.5.2 2. How FCFS Works

11.5.2.1 2.1 Basic Mechanism

- The disk head services requests **sequentially** based on their arrival time.
- No reordering or optimization is applied-requests are executed in the order they enter the queue.

11.5.2.2 2.2 Example Scenario

- **Given:**
 - A disk request queue with cylinder requests in the following order: **98, 183, 37, 122, 14, 124, 65, 67**
 - Initial disk head position: **Cylinder 53**
- **Sequence of Head Movements:**
 1. **53 -> 98** (Move from initial position to first request)
 2. **98 -> 183**
 3. **183 -> 37**
 4. **37 -> 122**
 5. **122 -> 14**
 6. **14 -> 124**
 7. **124 -> 65**
 8. **65 -> 67**

11.5.2.3 2.3 Calculation of Total Head Movement

- The total distance the disk head travels is the sum of absolute differences between consecutive cylinder positions.

Movement	Calculation	Distance (Cylinders)
53 -> 98	$98 - 53 = 45$	45
98 -> 183	$183 - 98 = 85$	85
183 -> 37	$183 - 37 = 146$	146
37 -> 122	$122 - 37 = 85$	85
122 -> 14	$122 - 14 = 108$	108
14 -> 124	$124 - 14 = 110$	110
124 -> 65	$124 - 65 = 59$	59
65 -> 67	$67 - 65 = 2$	2
Total Head Movement	Sum = 45 + 85 + 146 + 85 + 108 + 110 + 59 + 2	640 cylinders

11.5.3 3. Performance Analysis of FCFS

11.5.3.1 3.1 Advantages

- **Simplicity:**
 - Easy to implement-no complex algorithms or data structures required.
- **Fairness:**
 - Ensures **no starvation**-all requests are serviced in arrival order.

11.5.3.2 3.2 Disadvantages

- **Inefficiency in Head Movement:**
 - Does **not optimize** for minimal disk head movement.
 - May result in **large, unnecessary seek times** (e.g., moving from **122** -> **14** -> **124**).
- **Poor Performance for Random Requests:**
 - If requests are **widely spaced**, the total head movement increases significantly.
 - **Example:** The swing from **183** -> **37** (146 cylinders) is highly inefficient.

11.5.3.3 3.3 Comparison with Optimized Scheduling

- **Potential Improvement:**
 - If requests for **nearby cylinders** were grouped, total head movement could be **reduced**.
 - Example: Servicing **14, 37, 65, 67** in sequence would minimize long jumps.
- **Alternative Algorithms** (not covered in this lecture but implied):
 - **SSTF (Shortest Seek Time First)**
 - **SCAN (Elevator Algorithm)**
 - **C-SCAN (Circular SCAN)**
 - These algorithms aim to **minimize seek time** by reordering requests.

11.5.4 4. Key Takeaways

1. **FCFS is the simplest disk scheduling algorithm**, processing requests in arrival order.
2. **Fairness is guaranteed**, but **efficiency is compromised** due to unoptimized head movement.
3. **Total head movement can be excessive**, leading to **higher I/O latency**.
4. **Performance can be improved** by using algorithms that **minimize seek time** (e.g., SSTF, SCAN).

11.5.5 5. Conclusion

- FCFS is a **baseline algorithm** in disk scheduling, useful for understanding fundamental trade-offs between **fairness and efficiency**.
- While **simple and fair**, it is **not optimal** for high-performance storage systems where **minimizing seek time** is critical.
- Future lectures will explore **more advanced scheduling algorithms** that address these inefficiencies.

11.6 Introduction to Disk Management

11.6.1 1. Overview of Disk Management

Disk management is a critical component of **mass storage management** in operating systems. It encompasses processes that prepare disks for data storage, enable system booting, and handle defects to ensure **data integrity** and **system reliability**.

Key aspects covered in this lecture: - **Disk formatting** (low-level and high-level) - **Booting from disk** (bootstrap process) - **Bad-block recovery** (error handling and data recovery)

11.6.2 2. Disk Formatting

11.6.2.1 2.1 Low-Level (Physical) Formatting Before a disk can store data, it must undergo **low-level formatting**, which establishes the fundamental data structure for each **sector** (the smallest addressable unit on a disk).

11.6.2.1.1 2.1.1 Sector Structure Each sector consists of three components: 1. **Header** - Contains metadata, including: - **Sector number** (unique identifier) - **Synchronization marks** (for read/write alignment) 2. **Data Area** - Typically **512 bytes** (though customizable; common sizes: **256, 512, 1024 bytes**). 3. **Trailer** - Includes: - **Error-Correcting Code (ECC)** - Ensures **data integrity** by detecting and correcting errors.

11.6.2.1.2 2.1.2 Role of ECC (Error-Correcting Code)

- When data is **written** to a sector, the ECC is **calculated and stored** alongside the data.
- When data is **read**, the ECC is **recalculated** and compared with the stored value.
 - **Match** -> Data is intact.
 - **Mismatch** -> Errors are detected and **corrected** (if possible).

11.6.2.1.3 2.1.3 Manufacturing and Customization

- Most hard disks are **low-level formatted at the factory**.
- The process:
 - Tests the disk for defects.
 - Initializes **logical block numbering (LBN)** to map sectors, excluding defective ones.
- Manufacturers may allow **customization of sector size**:
 - Larger sectors (e.g., **1024 bytes**) reduce the number of sectors per track but **minimize overhead** (fewer headers/trailers -> more space for user data).

11.6.2.2 2.2 High-Level Formatting Before an OS can use a disk for file storage, it must perform **high-level formatting**, which involves two steps:

11.6.2.2.1 2.2.1 Partitioning

- The disk is divided into **one or more partitions** (groups of cylinders).
- Each partition is treated as a **separate logical disk**.
 - Example:
 - * **Partition 1**: OS executable code.
 - * **Partition 2**: User files.
- Enables **multi-boot systems** (e.g., dual-booting Windows and Linux).

11.6.2.2.2 2.2.2 File System Creation The OS writes **initial file system data structures**, including: - **Free/allocated space maps** (e.g., FAT, inode tables). - **Root directory** (initially empty). - **Boot control blocks** (for system startup).

11.6.2.2.3 2.2.3 Clustering

- To improve efficiency, file systems group **logical blocks** into larger units called **clusters** (or **allocation units**).
- Reduces fragmentation and overhead in metadata management.

11.6.2.2.4 2.2.4 Raw Disk Access

- Some OSes allow **raw I/O** (direct access to disk partitions **without** file system structures).
- Used by **specialized applications** (e.g., databases, custom storage systems).
- **Raw disk**: Treated as a **sequential array of logical blocks**.

11.6.3 3. Booting from Disk

11.6.3.1 3.1 Bootstrap Program

- A computer requires an **initial program** (bootstrap program) to:
 1. Initialize hardware.
 2. Load the **OS kernel** from disk into memory.
- The bootstrap process occurs in **two stages**:

11.6.3.1.1 3.1.1 Stage 1: ROM-Based Bootstrap

- Stored in **read-only memory (ROM)** (non-volatile, executes at power-up).
- Contains **minimal code** to:
 - Perform **hardware self-tests** (POST - Power-On Self-Test).
 - Locate and load the **full bootstrap program** from disk.

11.6.3.1.2 3.1.2 Stage 2: Disk-Based Bootstrap (Boot Block)

- The **full bootstrap program** is stored in the **boot block** (a reserved disk sector).
- A disk containing a boot partition is called a **boot disk** or **system disk**.

11.6.3.2 3.2 Boot Process in Windows The Windows boot process follows these steps:

1. Master Boot Record (MBR)

- Located in the **first sector** of the hard disk.
- Contains:
 - **Boot code** (initial instructions to start the OS).
 - **Partition table** (identifies the **boot partition**).
- The **ROM-based bootstrap** reads the MBR to locate the boot partition.

2. Boot Partition

- Contains:
 - **Operating system files**.
 - **Device drivers**.
- The system reads the **boot sector** from this partition to:
 - Load the **OS kernel**.
 - Initialize **subsystems** (e.g., memory management, I/O).

3. OS Initialization

- The kernel takes control, loads **system services**, and presents the **login screen**.

11.6.4 4. Bad-Block Recovery

11.6.4.1 4.1 Causes of Bad Blocks

- Disks are prone to **physical defects** (e.g., scratches, magnetic degradation).
- Bad blocks can occur due to:
 - **Manufacturing defects**.
 - **Wear and tear** over time.

- **Electrical/magnetic interference.**

11.6.4.2 4.2 Handling Bad Blocks

11.6.4.2.1 4.2.1 Detection

- **During formatting:** Bad blocks are **flagged as unusable**.
- **During operation:** Tools like Linux's `badblocks` command can **scan and lock** defective sectors.

11.6.4.2.2 4.2.2 Automatic Management (Modern Disks)

- Advanced disks maintain a **list of bad blocks** and use **spare sectors** for replacement.
- Two primary methods:

1. Sector Sparing

- A **bad sector** is replaced with a **spare sector** from a reserved pool.
- **Advantage:** No disruption to logical block numbering.
- **Disadvantage:** Limited by the number of spare sectors.

2. Sector Slipping

- All sectors **after the bad sector** are **shifted down** to skip the defective one.
- **Advantage:** No need for spare sectors.
- **Disadvantage:**
 - **Disrupts disk-scheduling optimizations** (OS expects contiguous sectors).
 - **Performance impact** due to non-sequential access.

11.6.4.3 4.3 Data Recovery from Bad Blocks

11.6.4.3.1 4.3.1 Soft Errors

- **Cause:** Temporary read/write failures (e.g., electrical noise).
- **Recovery:**
 - **ECC** can often **correct** minor errors.
 - Retrying the read operation may succeed.

11.6.4.3.2 4.3.2 Hard Errors

- **Cause:** Permanent physical damage.
- **Recovery:**
 - **Data loss** is likely if no backup exists.
 - **Manual intervention** required:
 - * Restore from **backups**.
 - * Use **data recovery tools** (e.g., `ddrescue` in Linux).

11.6.4.3.3 4.3.3 Best Practices

- **Regular backups** are **essential** to mitigate data loss.
- **SMART (Self-Monitoring, Analysis, and Reporting Technology)** can predict disk failures.

11.6.5 5. Summary and Key Takeaways

- **Disk formatting** is a **two-stage process**:
 - **Low-level:** Physical sector structure (ECC, headers, trailers).

- **High-level:** Partitioning and file system creation.
- **Booting** relies on:
 - **ROM-based bootstrap** (initial hardware initialization).
 - **Disk-based boot block** (loads the OS kernel).
- **Bad-block handling** includes:
 - **Detection** (formatting, badblocks).
 - **Recovery methods** (sector sparing/slipping).
 - **Data recovery** (ECC for soft errors, backups for hard errors).
- **Efficiency and reliability** depend on:
 - Proper **partitioning** and **clustering**.
 - **Regular backups** to prevent data loss.

11.7 Introduction

11.7.1 1. Overview of Memory Types in Computing Systems

- Computing systems utilize **two primary types of memory**:
 1. **Primary Memory (Main Memory)**
 - Acts as the computer's **short-term memory**.
 - Stores **data and instructions** actively used or processed by the **CPU**.
 - **Volatile**: Data is lost when power is turned off.
 - Examples: RAM (Random Access Memory), cache memory.
 2. **Secondary Memory (Auxiliary/Mass Storage)**
 - Serves as **long-term storage** for large volumes of data.
 - **Non-volatile**: Retains data even when power is off.
 - Enables **persistent access** to information over extended periods.
 - Examples: Hard drives (HDDs), solid-state drives (SSDs), CDs, DVDs, flash drives, cloud storage.

11.7.2 2. Definition of Mass Storage

- **Mass storage** refers to **high-capacity storage systems and devices** designed to:
 - Store **large amounts of data persistently and reliably**.
 - Support **modern computing environments** by providing space for data retention and retrieval.
 - Serve as a **critical component** for both individual and enterprise data management.

11.7.3 3. Key Characteristics of Mass Storage Devices

Mass storage devices exhibit the following essential traits:

1. **High Capacity**
 - Storage ranges from **gigabytes (GB)** to **petabytes (PB)**.
 - Accommodates growing data demands (e.g., multimedia, databases, scientific datasets).
2. **Data Persistence**
 - **Non-volatile**: Data remains intact **even when power is switched off**.
 - Ensures long-term retention without active power supply.
3. **Reliability**
 - Incorporates **error-checking mechanisms** (e.g., ECC - Error-Correcting Code).
 - Uses **redundancy techniques** (e.g., RAID, backups) to prevent data loss.
 - Designed for **durability** under continuous or intermittent use.
4. **Efficient Access Mechanisms**
 - Provides **fast and structured methods** for reading/writing data.
 - Optimized for **random or sequential access** depending on the technology (e.g., SSDs vs. HDDs).

11.7.4 4. Types of Mass Storage Devices

Mass storage devices are categorized based on their underlying technology. Below are the major types:

11.7.4.1 4.1 Magnetic Storage

- Uses **magnetic fields** to store data on rotating or stationary media.
- **Examples:**
 - **Hard Disk Drives (HDDs)**
 - * Store data on **spinning magnetic platters** using read/write heads.
 - * **Advantages:** High capacity, cost-effective, widely used.
 - * **Disadvantages:** Slower than SSDs (due to mechanical movement), susceptible to physical damage.
 - **Tape Drives**
 - * Used primarily for **large-scale archival storage** (e.g., backups, cold storage).
 - * **Advantages:** Extremely high capacity, low cost per GB.
 - * **Disadvantages:** Slow access times (sequential read/write), not suitable for frequent access.

11.7.4.2 4.2 Optical Storage

- Uses **laser technology** to read/write data on optical discs.
- **Examples:**
 - **Compact Discs (CDs)**
 - * Capacity: ~700 MB.
 - * Used for **software distribution, audio storage**.
 - **Digital Versatile Discs (DVDs)**
 - * Capacity: 4.7 GB (single-layer) to 17 GB (dual-layer, double-sided).
 - * Used for **video, software, and data storage**.
 - **Blu-ray Discs**
 - * Capacity: 25 GB (single-layer) to 128 GB (multi-layer).
 - * Used for **high-definition video and large-scale data archiving**.
- **Advantages:**
 - Portable, durable (resistant to magnetic interference).
 - Low cost per unit.
- **Disadvantages:**
 - Limited capacity compared to modern alternatives (e.g., SSDs).
 - Slower access speeds.
 - Declining use in favor of digital distribution and cloud storage.

11.7.4.3 4.3 Solid-State Storage

- Uses **flash memory technology** (NAND-based) with no moving parts.
- **Examples:**
 - **Solid-State Drives (SSDs)**
 - * Replace HDDs in computers for **faster performance**.
 - * **Advantages:**
 - **Speed:** Faster read/write operations (low latency).
 - **Durability:** No moving parts -> less prone to mechanical failure.
 - **Energy efficiency:** Lower power consumption.
 - * **Disadvantages:**
 - Higher cost per GB compared to HDDs.

- Limited write cycles (though modern SSDs mitigate this with wear-leveling).
- **Flash Drives (USB Drives, Thumb Drives)**
 - * Portable, plug-and-play storage.
 - * **Advantages:** Compact, reusable, high-speed data transfer.
 - * **Disadvantages:** Limited lifespan (finite write cycles), risk of loss/theft.

11.7.4.4 4.4 Network-Attached Storage (NAS)

- **Dedicated file storage devices** connected to a **network** (LAN or WAN).
- **Key Features:**
 - Allows **multiple users/systems** to access shared data simultaneously.
 - Often includes **RAID configurations** for redundancy.
 - Supports **remote access** via internet (with proper security).
- **Use Cases:**
 - Home media servers.
 - Small business data sharing.
 - Collaborative work environments.
- **Advantages:**
 - Centralized storage -> easier management.
 - Scalable (can add more drives as needed).
- **Disadvantages:**
 - Dependent on network speed.
 - Requires initial setup and maintenance.

11.7.4.5 4.5 Cloud Storage

- Stores data on **remote servers** accessed via the internet.
- **Key Features:**
 - **Scalability:** Pay-as-you-go models (e.g., GB/TB increments).
 - **Redundancy:** Data replicated across multiple servers/locations.
 - **Accessibility:** Available from **anywhere with an internet connection**.
 - **Built-in backups:** Automatic versioning and recovery options.
- **Examples:**
 - Google Drive, Dropbox, Microsoft OneDrive, Amazon S3.
- **Advantages:**
 - No physical hardware maintenance.
 - High availability and disaster recovery.
 - Collaboration features (e.g., real-time editing).
- **Disadvantages:**
 - **Security/privacy concerns** (data stored off-site).
 - **Recurring costs** (subscription-based).
 - **Dependence on internet connectivity**.

11.7.5 5. Applications of Mass Storage

Mass storage is integral to various domains, including:

1. Enterprise Data Management

- Stores **business-critical data:**
 - Databases (e.g., customer records, transactions).
 - Enterprise resource planning (ERP) systems.

- Ensures **data integrity and availability** for operations.
- 2. Personal Data Storage**
 - Individuals use mass storage for:
 - Documents (e.g., Word, PDF).
 - Multimedia (photos, videos, music).
 - Software applications.
 - 3. Backup and Disaster Recovery**
 - **Backups:** Regular copies of data to prevent loss from hardware failure, corruption, or cyberattacks.
 - **Disaster Recovery:** Restores systems/data after catastrophic events (e.g., fires, floods).
 - **Methods:**
 - Local backups (e.g., external HDDs, NAS).
 - Off-site backups (e.g., cloud storage, tape archives).
 - 4. Media Distribution**
 - Storage and distribution of **large media files:**
 - Movies, games, software updates.
 - Streaming services (e.g., Netflix, Spotify) rely on mass storage for content delivery.
 - 5. Scientific and Research Data**
 - Handles **vast datasets** generated by:
 - Genomics, astronomy, climate modeling.
 - Machine learning/AI training datasets.
 - Requires **high-capacity, high-speed storage** (e.g., SSDs, distributed storage systems).

11.7.6 6. Conclusion: Importance of Mass Storage

- Mass storage is a **fundamental component** of modern computing infrastructure.
- **Diverse technologies** (magnetic, optical, solid-state, NAS, cloud) cater to **different needs:**
 - **Cost vs. performance** (e.g., HDDs for bulk storage, SSDs for speed).
 - **Accessibility vs. security** (e.g., cloud for remote access, local NAS for control).
- **Future Trends:**
 - Advancements in **storage density** (e.g., 3D NAND, DNA storage).
 - Integration with **AI and automation** for smarter data management.
 - Growth of **hybrid storage solutions** (combining local and cloud storage).
- **Key Takeaway:** Understanding mass storage technologies enables **effective data management** and prepares for **emerging innovations** in storage systems.

11.8 Magnetic Disk

11.8.1 1. Introduction to Magnetic Disks

- **Definition:** Magnetic disks are **non-volatile memory** devices that provide **bulk storage** capabilities.
- **Key Property:** Retain data even when power is turned off (non-volatile).
- **Primary Use:** Secondary storage in computer systems due to high capacity and durability.

11.8.2 2. Physical Structure of a Magnetic Disk

11.8.2.1 2.1. Platters

- **Basic Element:** A **circular disk** (platter) made of **non-magnetic material.**
 - **Traditional Material:** Aluminium (historically used).
 - **Modern Material:** **Glass** (preferred due to **improved surface uniformity**).
- **Coating:** Covered with a **magnetizable material** (e.g., **iron oxide**) to store data magnetically.
- **Dimensions:**

- **Diameter Range:** **1.8 inches** (small drives, e.g., laptops) to **5.25 inches** (larger drives, e.g., desktops/servers).

11.8.2.2 2.2. Spindle and Rotation

- **Mounting:** Platters are mounted on a **rotatable spindle**.
- **Rotational Speed:**
 - **Range:** **5,400 RPM** (consumer drives) to **15,000 RPM** (high-performance drives, e.g., enterprise SSDs).
 - **Impact:** Higher RPM = **faster data access** but **increased heat and power consumption**.

11.8.2.3 2.3. Read/Write Head

- **Function:** Reads and records information on the platter.
- **Mechanism:**
 - Mounted on an **arm** that moves **radially** (inward/outward) across the platter.
 - **Positioning:** Can be placed over any **track** (concentric ring on the platter surface).

11.8.2.4 2.4. Multi-Platter Configuration (Hard Disks)

- **Structure:** Modern hard disks contain **multiple platters** stacked vertically.
- **Surfaces:** Each platter has **two surfaces** (top and bottom), each with its own **read/write head**.
- **Alignment:** Heads are **aligned vertically** to form **cylinders** (see Section 3.1).

11.8.3 3. Data Organization on Magnetic Disks

11.8.3.1 3.1. Tracks, Cylinders, and Sectors

- **Track:**
 - **Definition:** A **concentric ring** on a platter surface where data is stored.
 - **Example:** Track 0 (outermost), Track 1 (next inner), etc.
- **Cylinder:**
 - **Definition:** A **logical vertical alignment** of tracks across all platters.
 - * **Example:** Track 0 on **all surfaces** forms **Cylinder 0**.
 - * **Purpose:** Simplifies data access (all heads read/write the same cylinder simultaneously).
- **Sector:**
 - **Definition:** A **segment** of a track where data is stored.
 - **Separation:** Sectors are separated by **gaps** (see Section 3.2).
 - **Size Variability:**
 - * **Example 1:** Seagate Cheetah (450GB HDD) -> **512 bytes/sector**.
 - * **Example 2:** Toshiba (5TB HDD) -> **4,096 bytes/sector**.

11.8.3.2 3.2. Gaps Between Sectors and Tracks

- **Purpose of Sector Gaps:**
 - Allows the **read/write head** to **recognize the end of a sector**.
 - Prevents **data corruption** due to head misalignment.
- **Purpose of Track Gaps:**
 - Minimizes **magnetic interference** between adjacent tracks.
 - Reduces errors from **head misalignment**.

11.8.3.3 3.3. Disk Parameters (Examples)

Parameter	Seagate Cheetah (450GB)	Toshiba (5TB)
Sector Size	512 bytes	4,096 bytes
Number of Tracks	595,848	3,279,583
Rotational Speed	15,000 RPM	7,199 RPM

11.8.4 4. Disk Capacity Calculation

- **Definition:** Total **storage capacity** of a disk, measured in **GB (Gigabytes)** or **TB (Terabytes)**.
- **Formula:**

Disk Capacity = (Bytes per Sector) × (Average Sectors per Track) × (Tracks per Surface) × (Surfaces per Platter) × (Platters per Disk)

- **Breakdown of Components:**
 1. **Bytes per Sector:** Data stored in a single sector (e.g., 512B, 4KB).
 2. **Average Sectors per Track:** Number of sectors in a typical track.
 3. **Tracks per Surface:** Number of concentric tracks on one platter surface.
 4. **Surfaces per Platter:** Typically **2** (top and bottom).
 5. **Platters per Disk:** Number of platters stacked in the drive.

11.8.4.1 4.1. Example Calculation

- **Given Parameters:**
 - Bytes per sector = **512**
 - Sectors per track = **300**
 - Tracks per surface = **20,000**
 - Surfaces per platter = **2**
 - Platters per disk = **5**
- **Calculation:**
$$\begin{aligned}\text{Capacity} &= 512 \times 300 \times 20,000 \times 2 \times 5 \\ &= 512 \times 300 \times 20,000 \times 10 \\ &= 30,720,000,000 \text{ bytes} \\ &\approx 28.61 \text{ GB}\end{aligned}$$

11.8.5 5. Read/Write Operations

11.8.5.1 5.1. Process Overview

1. **Seek:** Move the **read/write head** radially to the **desired track**.
2. **Rotational Latency:** Wait for the **platter to rotate** until the **target sector** is under the head.
3. **Data Transfer:** Read/write data as the sector passes under the head.

11.8.5.2 5.2. Step-by-Step Example

- **Scenario:** Reading a sector (blue) on **Track 1**.
 1. **Seek:** Move the head to **Track 1**.
 2. **Rotation:** Rotate the platter **counter-clockwise** until the sector aligns with the head.
 3. **Reading:** Read bytes sequentially as the platter rotates.

- **New Request:** Reading a sector on the **last track**.
 1. **Seek:** Move the head to the **last track** (seek time incurred).
 2. **Rotational Latency:** Wait for the sector to align under the head.
 3. **Transfer:** Read the sector and transfer data.

11.8.5.3 5.3. Key Timing Metrics

- **Seek Time:**
 - **Definition:** Time taken to move the head to the **target track**.
 - **Dependent Factors:** Distance between tracks, head mechanism speed.
- **Rotational Latency:**
 - **Definition:** Time for the **target sector** to rotate under the head.
 - **Average Latency:** **Half the time for a full rotation** (e.g., 3ms for 15,000 RPM).
- **Data Transfer Time:**
 - **Definition:** Time to read/write the sector's data.
 - **Dependent Factors:** Rotational speed, sector size.

11.8.6 6. Summary of Key Concepts

11.8.6.1 6.1. Physical Components

- **Platters:** Glass/aluminium disks coated with magnetizable material.
- **Spindle:** Rotates platters at **5,400-15,000 RPM**.
- **Read/Write Head:** Moves radially to access tracks; aligned in cylinders for multi-platter disks.

11.8.6.2 6.2. Data Organization

- **Tracks:** Concentric rings on a platter surface.
- **Cylinders:** Vertical alignment of tracks across platters.
- **Sectors:** Subdivisions of tracks; separated by gaps to prevent errors.
- **Gaps:** Essential for sector/head alignment and magnetic interference reduction.

11.8.6.3 6.3. Capacity Calculation

- **Formula:** Product of bytes/sector, sectors/track, tracks/surface, surfaces/platter, and platters/disk.
- **Example:** 28.61 GB for given parameters.

11.8.6.4 6.4. Read/Write Process

- **Steps:** Seek -> Rotational Latency -> Data Transfer.
- **Timing Metrics:**
 - **Seek Time:** Head movement duration.
 - **Rotational Latency:** Sector alignment delay.
 - **Transfer Time:** Data read/write duration.

11.9 Magnetic Tapes

11.9.1 Introduction to Magnetic Tapes

- **Role in Mass Storage Management:** Magnetic tapes serve as a **secondary storage medium** in mass storage systems.
- **Historical Significance:** Among the **earliest forms of secondary storage**, predating many modern storage technologies.

- **Key Characteristics:**
 - **Permanence:** Data stored on magnetic tapes is **relatively permanent**, making them suitable for **long-term retention**.
 - **High Capacity:** Capable of storing **large quantities of data**, often scaling to **several terabytes**.
 - **Slow Access Time:** Significantly **slower access times** compared to **main memory** and **magnetic disks** due to their **sequential nature**.

11.9.2 Performance Limitations

- **Access Speed Comparison:**
 - **Random access** on magnetic tapes is **~1000 times slower** than on magnetic disks.
 - **Sequential access** is required to locate data, leading to delays (e.g., **minutes to reach the correct spot** on the tape).
- **Use Case Implications:**
 - **Not suitable for primary or frequently accessed storage** due to latency.
 - **Ideal for infrequently accessed data**, such as **backups** and **archives**.

11.9.3 Primary Applications of Magnetic Tapes

1. **Data Backup:**
 - **Reliable for long-term storage** of critical data.
 - **Cost-effective** for large-scale backups compared to disks or SSDs.
2. **Data Transfer:**
 - **Portable medium** for transferring large datasets between systems.
 - Historically used for **offline data distribution** (e.g., software distribution, database migrations).
3. **Archival Storage:**
 - **Durability:** Less prone to degradation over time compared to optical media (e.g., CDs/DVDs).
 - **Energy Efficiency:** Requires **no power** to maintain data integrity when not in use.

11.9.4 Functional Mechanics of Magnetic Tapes

11.9.4.1 Data Storage and Retrieval Process

- **Physical Structure:**
 - Data is stored on a **spool** of magnetic tape.
 - The tape **winds/unwinds** past a **read-write head** during operations.
- **Access Workflow:**
 1. **Positioning:** The tape drive **physically moves** the tape to the **correct location** (latency-intensive step).
 2. **Read/Write:** Once positioned, data transfer occurs at **speeds comparable to disk drives** (e.g., **140 MB/s** for LTO tapes).
- **Latency Bottleneck:**
 - **Sequential access** requires **linear scanning** to reach the target data block.

11.9.4.2 Capacity and Compression

- **Storage Capacity:**
 - **Modern tapes** can exceed **several terabytes** (e.g., **LTO-5 tapes store 1.5 TB uncompressed**).
 - **Compression:** Some tapes include **built-in hardware compression**, effectively **doubling storage capacity** (e.g., **3 TB compressed for LTO-5**).
- **Transfer Rates:**
 - High-end tapes achieve **up to 140 MB/s** (e.g., LTO technology).

11.9.5 Classification of Magnetic Tapes

11.9.5.1 By Physical Width Common tape widths include: - **4 mm** (e.g., Digital Audio Tape, DAT) - **8 mm** (e.g., AIT, Mammoth tapes) - **19 mm** - **1/4 inch** - **1/2 inch** (e.g., older reel-to-reel tapes)

11.9.5.2 By Technology Tapes are often categorized by their **technological standards**: 1. **LTO (Linear Tape-Open)**: - **Generations**: LTO-1 to LTO-9 (as of latest standards). - **Capacity**: Up to **1.5 TB (uncompressed)** and **3 TB (compressed)** for LTO-5. - **Transfer Rate**: Up to **140 MB/s**. - **Use Case**: Enterprise-level backup and archival. 2. **SDLT (Super Digital Linear Tape)**: - **Design Goal**: High-capacity, reliable **backup and archival** solutions. - **Features**: - Advanced error correction. - Higher durability compared to older linear tapes. - **Typical Applications**: Data centers, long-term storage for compliance (e.g., financial records).

11.9.6 Advantages of Magnetic Tapes

1. **High Capacity**:
 - Scalable to **multiple terabytes** per cartridge.
 - **Cost-per-GB** is lower than disks or SSDs for large datasets.
2. **Long-Term Storage**:
 - **Data persistence** for **decades** with proper handling.
 - **Resistant to obsolescence** (e.g., LTO standards are backward-compatible).
3. **Portability**:
 - **Physical transport** of large datasets without network dependencies.
4. **Energy Efficiency**:
 - **Zero power consumption** when not in use (unlike disks/SSDs).
5. **Security**:
 - **Air-gapped storage**: Immune to cyberattacks when offline.
 - **Write-once (WORM) options** for compliance (e.g., legal/financial archives).

11.9.7 Disadvantages of Magnetic Tapes

1. **Slow Access Time**:
 - **Sequential access** requires **minutes to locate data**.
 - **Unsuitable for real-time applications** (e.g., databases, OS operations).
2. **Inefficient for Random Access**:
 - **No direct addressing** of data blocks; must scan linearly.
3. **Mechanical Wear**:
 - **Physical degradation** over repeated use (e.g., tape stretching, head alignment issues).
4. **Limited Reusability**:
 - **Write cycles** are finite (though modern tapes support **thousands of passes**).
5. **Human Intervention**:
 - **Manual handling** required for loading/unloading tapes (unlike automated disk arrays).

11.9.8 Comparison with Other Storage Media

Feature	Magnetic Tapes	Magnetic Disks (HDDs)	Solid-State Drives (SSDs)
Access Speed	Very slow (sequential)	Fast (random access)	Very fast (random access)
Capacity	Very high (TBs per tape)	High (TBs per drive)	Moderate (TBs per drive)
Cost per GB	Very low	Low	High
Durability	High (decades)	Moderate (5-10 years)	Moderate (5-10 years)

Feature	Magnetic Tapes	Magnetic Disks (HDDs)	Solid-State Drives (SSDs)
Portability	High (physical transport)	Low (bulky, fragile)	Moderate (smaller form factor)
Power Consumption	None (when idle)	High (spinning disks)	Low
Use Case	Backup, archives, transfer	Primary storage, databases	Primary storage, caching

11.9.9 Modern Relevance and Future Outlook

- **Niche Dominance:**
 - Despite **declining use in general computing**, tapes remain **critical for**:
 - * **Enterprise backups** (e.g., banks, governments).
 - * **Scientific data archives** (e.g., CERN, NASA).
 - * **Cold storage** (e.g., cloud providers like AWS Glacier use tape-based systems).
- **Technological Advancements:**
 - **LTO-9** (2021) supports **18 TB uncompressed** and **45 TB compressed**.
 - **Roadmap:** LTO-10 and beyond aim for **36 TB+ capacities**.
- **Hybrid Systems:**
 - **Tape + Disk/Cloud:** Modern solutions combine tapes for **long-term storage** with disks/SSDs for **active data**.
- **Challenges:**
 - **Competition from optical storage** (e.g., Blu-ray archives) and **DNA data storage** (emerging tech).
 - **Declining expertise** in tape management among newer IT professionals.

11.9.10 Summary of Key Points

- **Definition:** Magnetic tapes are **sequential-access, high-capacity** secondary storage media.
- **Strengths:**
 - **High capacity, low cost, long-term durability, portability.**
- **Weaknesses:**
 - **Slow access, mechanical limitations, poor random access.**
- **Primary Uses:**
 - **Backup, archival, data transfer.**
- **Technologies:**
 - **LTO** (high performance) and **SDLT** (reliability-focused).
- **Future:** Continued relevance in **large-scale archives** and **hybrid storage systems**.

11.10 Module Summary

11.10.1 1. Introduction to Mass Storage Devices

11.10.1.1 1.1 Role of Disk Drives in Computing Systems

- Disk drives serve as the **primary secondary storage I/O devices** in most computer systems.
- They provide **persistent storage** for data, programs, and system files beyond volatile memory (RAM).

11.10.1.2 1.2 Types of Mass Storage Devices Three major categories of secondary storage devices are discussed: 1. **Magnetic Disks (HDDs - Hard Disk Drives)** - Traditional spinning disks with read/write heads. - Data stored magnetically on rotating platters. 2. **Magnetic Tapes** - Sequential-access storage medium. - Used for archival and backup due to high capacity and low cost. 3. **Solid-State Disks (SSDs) - Increasingly dominant** in

modern systems due to speed and reliability. - Use **flash memory** (NAND-based) with no moving parts. - Offer **faster access times** and **lower latency** compared to HDDs.

11.10.1.3 1.3 Logical Organization of Disk Storage

- Disks are organized as **large arrays of logical disk blocks**.
- Standard block size is typically **512 bytes** (though modern systems may use **4KB blocks**).
- Each block is addressable via a **logical block number (LBN)**.

11.10.2 2. Disk Connection Methods

Disk drives can be connected to computer systems in two primary ways: 1. **Local I/O Ports** - Directly attached to the host system (e.g., SATA, NVMe, SAS). - Provides **low-latency access** but limited scalability. 2. **Network Connections** - Accessed over a network (e.g., NAS - Network-Attached Storage, SAN - Storage Area Network). - Enables **shared storage** across multiple systems. - Useful for **distributed systems** and **enterprise storage solutions**.

Flexibility in Configurations: - The choice between local and network-attached storage depends on: - **System requirements** (performance, scalability). - **Architectural constraints** (single-user vs. multi-user environments).

11.10.3 3. Sources of Disk I/O Requests

Disk I/O operations originate from two key OS components: 1. **File System** - Manages **persistent data storage** (files, directories). - Issues read/write requests for file operations (e.g., opening, saving, deleting files). 2. **Virtual Memory System** - Handles **paging and swapping** for memory management. - Requests disk I/O for **page-ins** (loading from disk to RAM) and **page-outs** (saving RAM to disk).

Request Specification: - Each I/O request specifies: - **Logical block number (LBN)** - Exact location on disk. - **Operation type** (read/write). - **Data size** (number of blocks to transfer).

11.10.4 4. Disk Scheduling Algorithms

Efficient disk scheduling is critical for **optimizing performance** by reducing **seek time** and **rotational latency**.

11.10.4.1 4.1 Key Performance Metrics

- **Effective Bandwidth** - Throughput of data transfer.
- **Average Response Time** - Time taken to complete an I/O request.
- **Variance in Response Time** - Consistency of performance across requests.

11.10.4.2 4.2 Common Disk Scheduling Algorithms

Algorithm	Description	Pros	Cons
FCFS (First-Come, First-Served)	Processes requests in order of arrival .	Simple, fair.	High variance in response time (inefficient for random access).
SSTF (Shortest Seek Time First)	Selects the request closest to the current head position .	Reduces seek time.	May cause starvation for distant requests.

Algorithm	Description	Pros	Cons
SCAN (Elevator Algorithm)	Moves the disk head back and forth , servicing requests in its path.	Balances fairness and efficiency.	Long wait times at the ends of the disk.
C-SCAN (Circular SCAN)	Similar to SCAN but wraps around after reaching the end.	More uniform response times.	Still suffers from end-of-disk delays.

11.10.4.3 4.3 Impact on Magnetic Disks vs. SSDs

- **Magnetic Disks (HDDs):**
 - Performance **highly dependent** on scheduling due to **mechanical delays** (seek time, rotational latency).
 - **SCAN/C-SCAN** often preferred for sequential access patterns.
- **Solid-State Disks (SSDs):**
 - **No moving parts** -> **consistent performance** regardless of scheduling.
 - **FCFS often sufficient** due to negligible seek/rotational delays.
 - Advanced algorithms (e.g., **wear-leveling**) handle internal optimization.

11.10.5 5. Operating System's Role in Disk Management

The OS performs several critical functions to ensure efficient disk usage:

11.10.5.1 5.1 Low-Level Formatting

- **Divides the disk into sectors** (physical blocks).
- **Initializes disk structures** for data storage.
- Typically done during **manufacturing** or **disk initialization**.

11.10.5.2 5.2 Partitioning

- **Divides the disk into logical partitions** (e.g., C:, D: in Windows).
- Each partition can:
 - Host a **separate file system**.
 - Be used for **different OS installations** (dual-boot systems).

11.10.5.3 5.3 File System Establishment

- **Formats partitions** with a file system (e.g., NTFS, ext4, FAT32).
- **Manages:**
 - **Metadata** (inodes, directory structures).
 - **Block allocation** for files.

11.10.5.4 5.4 Block Allocation & System Boot

- **Allocates blocks** for system files (e.g., bootloader, OS kernel).
- Ensures **fast boot times** by optimizing placement of critical files.

11.10.5.5 5.5 Handling Corrupted Blocks

- **Bad block management:**
 - **Isolates defective sectors** to prevent data corruption.

- **Logically replaces** bad blocks with **spare blocks** (remapping).
- **SMART (Self-Monitoring, Analysis, and Reporting Technology)** helps predict disk failures.

11.10.6 6. SWAP Space Management

11.10.6.1 6.1 Role of SWAP Space

- Used by the **virtual memory system** for **paging** (moving data between RAM and disk).
- Acts as an **overflow area** when physical RAM is full.

11.10.6.2 6.2 Optimization Techniques

- **Bypasses traditional file systems** for paging I/O to:
 - **Reduce overhead** (no file system metadata processing).
 - **Improve speed** (direct block-level access).
- **Contiguous allocation** of SWAP space enhances performance.

11.10.7 7. Redundant Array of Independent Disks (RAID)

RAID provides **reliability, performance, and fault tolerance** through disk redundancy.

11.10.7.1 7.1 Key RAID Levels

RAID Level	Description	Benefits	Drawbacks
RAID 0 (Striping)	Data split across multiple disks (no redundancy).	High performance (parallel reads/writes).	No fault tolerance (single disk failure = data loss).
RAID 1 (Mirroring)	Duplicate data on two disks.	Full redundancy, fast read performance.	50% storage overhead (only half capacity usable).
RAID 5 (Striping + Parity)	Data + parity distributed across ≥ 3 disks .	Balances performance and redundancy.	Write penalty (parity calculation slows writes).
RAID 6 (Double Parity)	Extends RAID 5 with two parity blocks .	Survives two disk failures .	Higher overhead, slower writes.
RAID 10 (1+0)	Mirroring + Striping (minimum 4 disks).	High performance + full redundancy.	50% storage overhead .

11.10.7.2 7.2 Benefits of RAID

- **Reliability:** Automatic recovery from disk failures.
- **Performance:** Parallel I/O operations improve throughput.
- **Scalability:** Can expand storage by adding disks.

11.10.7.3 7.3 Use Cases

- **RAID 0:** High-performance applications (e.g., gaming, video editing).
- **RAID 1/10:** Critical systems requiring **fault tolerance** (e.g., databases, servers).
- **RAID 5/6:** **Balanced** solutions for general-purpose storage.

11.10.8 8. Fragmentation and Performance Issues

11.10.8.1 8.1 Disk Fragmentation

- **External Fragmentation:** Free space broken into small, non-contiguous blocks.
 - **Impact:** Slows down read/write operations (more seek time).
- **Internal Fragmentation:** Wasted space within allocated blocks.
 - **Impact:** Reduces storage efficiency.

11.10.8.2 8.2 Mitigation Strategies

- **Defragmentation (for HDDs):**
 - Reorganizes data to **reduce seek time**.
 - **Not needed for SSDs** (no mechanical delays).
- **File System Optimization:**
 - **Extents** (e.g., ext4, NTFS) reduce fragmentation by allocating contiguous blocks.
 - **Journaling** (e.g., ext4, ZFS) prevents corruption after crashes.

11.10.9 9. Conclusion: Importance of Efficient Disk Management

- **Performance Optimization:**
 - Proper scheduling, RAID, and SWAP management **maximize system responsiveness**.
- **Reliability & Fault Tolerance:**
 - RAID and bad block handling **prevent data loss**.
- **Scalability:**
 - Network-attached storage and partitioning **support growing data needs**.

11.10.9.1 9.1 Key Takeaways

1. **Disk scheduling** (FCFS, SSTF, SCAN, C-SCAN) impacts HDD performance but is less critical for SSDs.
2. **RAID configurations** provide trade-offs between **speed, redundancy, and cost**.
3. **SWAP space management** is crucial for **virtual memory efficiency**.
4. **Fragmentation control** (defragmentation, extents) maintains **optimal disk performance**.
5. **OS disk management** (formatting, partitioning, bad block handling) ensures **long-term reliability**.

Final Note: Understanding these concepts is **essential** for designing **high-performance, reliable storage systems** in modern computing environments.

11.11 RAID Structure

11.11.1 Introduction to RAID

- **Definition:** RAID stands for **Redundant Array of Independent Disks** (or **Inexpensive Disks**).
- **Purpose:** Combines multiple physical disk drives into a **single logical unit** to improve:
 - **Performance** (faster read/write operations).
 - **Redundancy** (data protection against disk failures).
- **Key Concept:** Data is **distributed** across multiple disks rather than stored on a single disk.
- **Striping:** The process of dividing data into **blocks (stripes)** and writing them across multiple disks.

11.11.2 Why Use Redundancy?

- **Redundant disks** store:
 - **Data copies** (mirroring).

- **Error detection/correction codes** (e.g., **parity information**).
- **Benefit:** Ensures **data recoverability** in case of disk failure.

11.11.3 RAID Levels Overview

There are **seven standard RAID levels**, each with unique characteristics.

11.11.3.1 RAID 0 (Striping)

- **Not a true RAID** (no redundancy).
- **Minimum disks required:** 2.
- **Mechanism:**
 - Data is split into **stripes** (e.g., Strip 0, Strip 1).
 - Stripes are written **sequentially across disks** (e.g., Strip 0 -> Disk 1, Strip 1 -> Disk 2).
- **Performance:**
 - **High read/write speeds** (parallel I/O operations).
 - **I/O load is distributed** across multiple disks.
- **Advantages:**
 - Simple design.
 - Easy to implement.
 - Improved performance.
- **Disadvantages:**
 - **No fault tolerance** (failure of **one disk** destroys the entire array).
- **Applications:**
 - Video production/editing.
 - Image editing.

11.11.3.2 RAID 1 (Mirroring)

- **Mechanism:**
 - **100% data duplication** (each logical stripe is mirrored to a second disk).
 - If there are **n data disks**, **n redundant disks** are required.
- **Fault Tolerance:**
 - **High redundancy** (data remains intact if one disk fails).
 - **No rebuild needed** (just copy data from the redundant disk to a replacement).
- **Advantages:**
 - **100% data redundancy**.
 - **High availability** (critical for fault-free systems).
- **Disadvantages:**
 - **Storage capacity is halved** (highest disk overhead).
 - **Expensive** (requires double the disks).
- **Applications:**
 - Critical data storage (accounting, payroll, financial systems).
 - Systems requiring **high availability**.

11.11.3.3 RAID 2 (Bit-Level Striping with Hamming Code)

- **Mechanism:**
 - **Bit-level striping** (data split at the bit level).
 - Uses **Hamming code** for **error correction**.
 - Requires **3 extra disks** for every **4 data disks**.

- **Advantages:**
 - **Hardware-level error correction.**
- **Disadvantages:**
 - **Complex and expensive.**
 - **Rarely used** in practice (historical significance only).

11.11.3.4 RAID 3 (Byte-Level Striping with Dedicated Parity Disk)

- **Mechanism:**
 - **Byte-level striping** (data split at the byte level).
 - **Dedicated parity disk** for error correction.
- **Performance:**
 - Good for **large sequential data transfers.**
 - **Parity disk bottleneck** (limits performance if parity disk fails).
- **Disadvantages:**
 - **Not commonly used** (better alternatives exist).
- **Applications:**
 - Large continuous data streams.

11.11.3.5 RAID 4 (Block-Level Striping with Dedicated Parity Disk)

- **Mechanism:**
 - **Block-level striping** (data split into fixed-size blocks).
 - **Dedicated parity disk** (like RAID 3, but at block level).
- **Performance:**
 - **Good read performance** (parallel reads).
 - **Write bottleneck** (parity disk slows writes).
- **Disadvantages:**
 - **Parity disk is a single point of failure.**
- **Applications:**
 - **Read-heavy databases.**

11.11.3.6 RAID 5 (Block-Level Striping with Distributed Parity)

- **Mechanism:**
 - **Block-level striping with distributed parity** (parity spread across all disks).
 - **No single parity disk bottleneck.**
- **Fault Tolerance:**
 - Survives **one disk failure.**
- **Performance:**
 - **Balanced read/write performance.**
 - **Slower writes** (parity calculation overhead).
- **Disadvantages:**
 - **Complex rebuild process.**
- **Applications:**
 - **General-purpose servers.**
 - **Enterprise storage.**

11.11.3.7 RAID 6 (Double Parity RAID)

- **Mechanism:**

- **Two parity blocks** (P and Q, using **Reed-Solomon algorithm**).
- **Distributed parity** (like RAID 5, but with extra redundancy).
- **Fault Tolerance:**
 - Survives **two simultaneous disk failures**.
- **Disadvantages:**
 - **Higher write overhead.**
 - **More complex** than RAID 5.
- **Applications:**
 - **High-availability systems.**
 - **Large-scale storage.**

11.11.4 Summary of RAID Levels

RAID Level	Key Feature	Redundancy	Performance	Use Case
RAID 0	Striping (no redundancy)	[X] No	* Very High	Video/image editing
RAID 1	Mirroring (100% redundancy)	[OK] Yes	* High (read)	Critical data (financial systems)
RAID 2	Bit-level striping + Hamming code	[OK] Yes	* Low (complex)	Rarely used (historical)
RAID 3	Byte-level striping + dedicated parity	[OK] Yes	* Moderate (bottleneck)	Large sequential data
RAID 4	Block-level striping + dedicated parity	[OK] Yes	* Good read, slow write	Read-heavy databases
RAID 5	Block-level striping + distributed parity	[OK] Yes (1 disk)	* Balanced	General-purpose servers
RAID 6	Double parity (Reed-Solomon)	[OK] Yes (2 disks)	* Slower writes	High-availability systems

11.11.5 Conclusion

- RAID provides **scalable solutions** for **performance** and **data redundancy**.
- **Choosing the right RAID level** depends on:
 - **Fault tolerance needs.**
 - **Performance requirements.**
 - **Budget constraints.**
- **Critical for designing robust storage systems** in enterprise and high-availability environments.

11.12 SCAN Disk Scheduling Algorithm

11.12.1 Introduction to SCAN Algorithm

- **Definition:**
 - The **SCAN algorithm** (also called the **elevator algorithm**) is a disk scheduling method used to optimize the movement of the disk arm (read-write head) across disk cylinders.
 - The name “elevator algorithm” is derived from its similarity to how an elevator services requests: moving in one direction, servicing requests along the way, then reversing direction at the end.
- **Core Mechanism:**
 - The disk arm starts at one end of the disk (e.g., cylinder 0 or the highest-numbered cylinder) and moves toward the opposite end, servicing all pending requests in its path.

- Upon reaching the opposite end, the arm reverses direction and continues servicing requests in the new direction.
- This back-and-forth scanning ensures that **all requests are eventually serviced** without indefinite postponement.
- **Key Characteristics:**
 - **Continuous Scanning:** The arm scans the disk in a linear, bidirectional motion.
 - **Fairness:** Prevents starvation by ensuring no request is ignored indefinitely.
 - **Efficiency:** Reduces **seek time** (time taken for the disk arm to move to the target cylinder) by servicing requests in a systematic order.

11.12.2 How the SCAN Algorithm Works

11.12.2.1 Step-by-Step Operation

1. **Initialization:**
 - The disk arm starts at a given position (e.g., cylinder *X*).
 - The direction of movement (toward higher or lower cylinder numbers) is predetermined.
2. **Servicing Requests:**
 - As the arm moves, it services **all pending requests** in its current direction.
 - Requests are serviced **only when the arm reaches their cylinder** (no out-of-order servicing).
3. **Direction Reversal:**
 - Upon reaching the **end of the disk** (either cylinder 0 or the maximum cylinder), the arm reverses direction.
 - The process repeats, servicing requests in the new direction.
4. **Termination:**
 - The algorithm continues indefinitely, ensuring all requests are eventually handled.

11.12.2.2 Example Scenario

- **Disk Configuration:**
 - Total cylinders: **200** (numbered **0 to 199**).
 - Initial head position: **Cylinder 53**.
 - Direction of movement: **Toward cylinder 0 (decreasing order)**.
- **Request Queue** (8 requests):
 - 37, 14, 65, 67, 98, 122, 124, 183.
- **Service Order:**
 1. **First Pass (Moving toward cylinder 0):**
 - Service **37** (move from 53 -> 37: **16 cylinders**).
 - Service **14** (move from 37 -> 14: **23 cylinders**).
 - Reach **0** (move from 14 -> 0: **14 cylinders**).
 2. **Direction Reversal (Moving toward cylinder 199):**
 - Service **65** (move from 0 -> 65: **65 cylinders**).
 - Service **67** (move from 65 -> 67: **2 cylinders**).
 - Service **98** (move from 67 -> 98: **31 cylinders**).
 - Service **122** (move from 98 -> 122: **24 cylinders**).
 - Service **124** (move from 122 -> 124: **2 cylinders**).
 - Service **183** (move from 124 -> 183: **59 cylinders**).
- **Total Head Movement:**
 - Sum of all movements: **16 + 23 + 14 + 65 + 2 + 31 + 24 + 2 + 59 = 236 cylinders**.

11.12.3 Advantages of the SCAN Algorithm

1. **Reduced Seek Time:**
 - By servicing requests in a linear path, the algorithm **minimizes unnecessary arm movements**, leading to lower average seek times compared to random-order algorithms (e.g., FCFS).
2. **Prevention of Starvation:**
 - Unlike **Shortest Seek Time First (SSTF)**, which may starve requests far from the current head position, SCAN ensures **all requests are serviced** in a finite time.
3. **Uniform Wait Times:**
 - Requests are serviced in a **predictable order**, reducing variance in wait times across different requests.
 - Requests at the **ends of the disk** (where the arm reverses) experience the longest waits but are still guaranteed service.
4. **Fairness:**
 - No request is indefinitely postponed, making SCAN **more equitable** than algorithms like FCFS or SSTF.

11.12.4 Comparison with Other Disk Scheduling Algorithms

Algorithm	Mechanism	Advantages	Disadvantages
FCFS	Serves requests in arrival order.	Simple to implement.	High seek times; inefficient for distant requests.
SSTF	Serves the nearest request first.	Minimizes seek time per request.	Starvation for distant requests.
SCAN	Linear back-and-forth servicing.	Balanced seek time; prevents starvation.	Higher wait times for middle cylinders.
C-SCAN	Circular version of SCAN.	More uniform wait times.	Longer response for requests near the end.

11.12.4.1 Key Comparisons

1. **SCAN vs. FCFS (First Come, First Served):**
 - FCFS may lead to **high seek times** if requests are scattered randomly across the disk.
 - SCAN optimizes seek time by servicing requests in a **systematic linear path**.
2. **SCAN vs. SSTF (Shortest Seek Time First):**
 - SSTF can cause **starvation** for requests far from the current head position.
 - SCAN ensures **fairness** by servicing all requests in a predictable manner.
3. **SCAN vs. C-SCAN (Circular SCAN):**
 - C-SCAN is a variant where the arm **only moves in one direction** and wraps around to the start, providing more uniform wait times.
 - SCAN may have **higher wait times for middle cylinders** due to bidirectional movement.

11.12.5 Limitations of the SCAN Algorithm

1. **Higher Wait Times for Middle Cylinders:**
 - Requests near the **center of the disk** may experience longer wait times because the arm prioritizes servicing requests at the ends first.
2. **Overhead at Reversal Points:**
 - The arm must **travel the full length of the disk** before reversing, which can introduce delays for requests near the reversal points.
3. **Not Optimal for All Workloads:**

- If requests are **clustered in one region**, SCAN may still incur unnecessary movement compared to algorithms like **LOOK** (which stops at the last request in a direction).

11.12.6 Practical Implications

- **Use Cases:**
 - Ideal for **general-purpose systems** where fairness and predictable performance are critical.
 - Suitable for **environments with a uniform distribution of requests** across the disk.
- **Optimizations:**
 - **LOOK Algorithm:** A variant of SCAN where the arm reverses direction at the **last request** in its current path (rather than the disk end), reducing unnecessary movement.
 - **C-LOOK:** Combines C-SCAN and LOOK for further optimization.

11.12.7 Summary of Key Points

1. **Definition:** SCAN is a disk scheduling algorithm that services requests in a linear, back-and-forth motion, similar to an elevator.
2. **Operation:**
 - The disk arm moves from one end to the other, servicing requests in its path.
 - Reverses direction upon reaching the end.
3. **Example Calculation:**
 - Total head movement for the given scenario: **236 cylinders**.
4. **Advantages:**
 - Reduces seek time.
 - Prevents starvation.
 - Provides uniform wait times.
5. **Comparisons:**
 - More efficient than FCFS.
 - Fairer than SSTF.
 - Less uniform than C-SCAN but simpler to implement.
6. **Limitations:**
 - Middle cylinders may experience longer waits.
 - Full disk traversal before reversal can introduce delays.

11.12.8 Conclusion

The SCAN algorithm strikes a **balance between efficiency and fairness** in disk scheduling. By systematically servicing requests in a linear path and reversing direction at the disk ends, it **minimizes seek time** while ensuring **no request is starved**. Understanding its operation through examples (e.g., the 200-cylinder disk) highlights its **predictability and uniformity**, making it a robust choice for many operating systems.

11.13 Solid State Disks

11.13.1 1. Introduction to Memory Types

11.13.1.1 1.1 Semiconductor Memories

- **Characteristics:**
 - Designed for **high-speed performance**.
 - **Lower storage capacity** compared to magnetic memories.
- **Common Applications:**
 - Used in **cache memory** and **main memory (RAM)**.

- Relies on **electronic circuits** for data storage and retrieval.

11.13.1.2 1.2 Magnetic Memories

- **Characteristics:**
 - **Slower performance** due to **mechanical components** (e.g., spinning disks, read/write heads).
 - **Higher storage capacity** compared to semiconductor memories.
- **Common Applications:**
 - Traditionally used in **hard disk drives (HDDs)**.

11.13.1.3 1.3 Key Comparison

Feature	Semiconductor Memory	Magnetic Memory
Speed	Faster (electronic)	Slower (mechanical)
Capacity	Lower	Higher
Mechanical Parts	None	Present (disks, heads)
Energy Consumption	Lower	Higher

11.13.2 2. The Need for Faster High-Capacity Memory

11.13.2.1 2.1 Performance Bottleneck in Magnetic Disks

- **Mechanical limitations** (seek time, rotational latency) slow down data access.
- **Question:** How can we improve access speed for high-capacity storage while retaining durability?

11.13.2.2 2.2 Solution: Solid State Disks (SSDs)

- **Definition:** SSDs are **non-volatile storage devices** that use **flash memory technology** (electrically programmable).
- **Purpose:** Serve as a **faster alternative** to traditional magnetic disks (HDDs).

11.13.3 3. Advantages of SSDs Over Magnetic Disks

11.13.3.1 3.1 Performance Benefits

- **Faster I/O Operations:**
 - **10x faster** than traditional HDDs.
 - **No seek time or rotational latency** (unlike HDDs, which require physical movement of read/write heads).
- **Lower Access Times & Latency:**
 - Data retrieval is **instantaneous** due to electronic access.

11.13.3.2 3.2 Physical & Operational Benefits

- **No Moving Parts:**
 - **Quieter operation** (no spinning disks or moving heads).
 - **Cooler operation** (less heat generation).
- **Energy Efficiency:**
 - **Lower power consumption** (no energy spent on moving mechanical components).
 - Referred to as “**green devices**” due to reduced energy usage.

11.13.3.3 3.3 Reliability & Durability

- **Resistance to Physical Shock:**
 - Less prone to damage from drops or vibrations (unlike HDDs).
- **Environmental Benefits:**
 - Reduced energy consumption aligns with **sustainable computing** practices.

11.13.4 4. Connectivity Options for SSDs

SSDs can be connected to systems via: 1. **USB (Universal Serial Bus):** - Common for **external SSDs** (portable storage). 2. **SATA (Serial Advanced Technology Attachment):** - Standard interface for **internal SSDs** in desktops/laptops. - Provides **high-speed data transfer** between the SSD and motherboard.

11.13.5 5. Internal Structure of SSDs

11.13.5.1 5.1 Block and Page Organization

- An SSD consists of:
 - **B blocks** (indexed from **Block 0** to **Block B-1**).
 - Each block contains **P pages** (indexed from **Page 0** to **Page P-1**).
- **Typical Page Size: 512 bytes to 4 KB.**
- **Pages per Block: 32 to 128 pages.**

11.13.5.2 5.2 Data Read/Write Mechanics

- **Read Operation:**
 - Data is read in **page-sized units**.
- **Write Operation:**
 - A page can **only be written after its entire block is erased**.
 - **Constraint: Erase-before-write** requirement slows down write operations if blocks are not pre-erased.

11.13.5.3 5.3 Wear-Out Mechanism

- **Lifespan Limitation:**
 - A block **wears out after ~100,000 write cycles** (varies by SSD model).
 - **Reason:** Flash memory cells degrade with repeated erase/write operations.
- **Implications:**
 - **Performance degradation** over time as blocks wear out.
 - **Eventual memory failure** after exceeding write limits.

11.13.6 6. Practical Issues with SSDs

11.13.6.1 6.1 Performance Slowdown Over Time

- **Cause:**
 - As the SSD fills up, **garbage collection** (reclaiming unused blocks) becomes more frequent, increasing latency.
 - **Fragmentation** of free space reduces write efficiency.
- **Mitigation:**
 - **TRIM command** (allows OS to inform SSD which blocks are no longer in use, improving garbage collection).

11.13.6.2 6.2 Memory Wear-Out

- **Cause:**
 - **Finite write endurance** (~100,000 cycles per block).
 - **Wear leveling** (distributing writes evenly across blocks) extends lifespan but does not eliminate the issue.
- **Consequences:**
 - **Unusable memory** after exceeding write limits.
 - **Data loss risk** if critical blocks fail.

11.13.7 7. Summary: Why SSDs Are Preferred Despite Limitations

11.13.7.1 7.1 Key Advantages

Feature	SSD	HDD
Speed	10x faster (no mechanical delay)	Slower (seek/rotational latency)
Energy Use	Low (no moving parts)	High (spinning disks)
Noise/Heat	Silent & cool	Noisy & generates heat
Durability	Shock-resistant	Vulnerable to physical damage
Environmental Impact	“Green” (low power)	Higher power consumption

11.13.7.2 7.2 Persistent Challenges

- **Performance degradation** over time (due to garbage collection and wear).
- **Limited write endurance** (blocks wear out after repeated use).

11.13.7.3 7.3 Conclusion

- Despite **wear-out and slowdown issues**, SSDs are **preferred** for:
 - **High-speed applications** (e.g., OS boot drives, databases).
 - **Energy-efficient computing** (laptops, data centers).
 - **Reliability in harsh environments** (e.g., mobile devices, embedded systems).

11.14 SSTF Disk Scheduling Algorithm

11.14.1 1. Introduction to SSTF (Shortest Seek Time First)

- **Definition:** SSTF (Shortest Seek Time First) is a **disk scheduling algorithm** that prioritizes servicing the **request closest to the current head position** to minimize **seek time**.
- **Core Principle:** The algorithm selects the next request based on the **shortest distance** the disk head must travel from its current position.
- **Objective:** Reduce **total head movement**, thereby improving disk I/O performance.

11.14.2 2. How SSTF Works

11.14.2.1 2.1 Basic Mechanism

- The disk head services the **nearest pending request** first.
- After completing a request, the algorithm **re-evaluates** the remaining requests and selects the next closest one.
- This process repeats until all requests are serviced.

11.14.2.2 2.2 Key Characteristics

- **Dynamic Selection:** Unlike FCFS (First-Come, First-Served), SSTF does not follow a strict queue order.
- **Seek Time Optimization:** Minimizes the **average seek time** by always choosing the closest request.
- **Non-Preemptive:** Once a request is selected, the head moves to its track and completes the operation before considering the next request.

11.14.3 3. Example Walkthrough

11.14.3.1 3.1 Given Scenario

- **Disk Request Queue:** [98, 183, 37, 122, 2, 14, 124, 65, 67]
- **Initial Head Position:** Cylinder 53

11.14.3.2 3.2 Step-by-Step Execution

1. **Initial Position:** Head at 53.
 - **Nearest Requests:**
 - **Right Side:** 65 (distance = $65 - 53 = 12$)
 - **Left Side:** 37 (distance = $53 - 37 = 16$)
 - **Decision:** Move to 65 (shorter distance).
 - **Head Movement:** 53 → 65 (12 cylinders).
2. **Head at 65:**
 - **Pending Requests:** [98, 183, 37, 122, 2, 14, 124, 67]
 - **Nearest Request:** 67 (distance = $67 - 65 = 2$).
 - **Head Movement:** 65 → 67 (2 cylinders).
3. **Head at 67:**
 - **Pending Requests:** [98, 183, 37, 122, 2, 14, 124]
 - **Nearest Request:** 37 (distance = $67 - 37 = 30$).
 - (Other distances: 14 → 53, 98 → 31, 122 → 55, etc.)
 - **Head Movement:** 67 → 37 (30 cylinders).
4. **Head at 37:**
 - **Pending Requests:** [98, 183, 122, 2, 14, 124]
 - **Nearest Request:** 14 (distance = $37 - 14 = 23$).
 - **Head Movement:** 37 → 14 (23 cylinders).
5. **Head at 14:**
 - **Pending Requests:** [98, 183, 122, 2, 124]
 - **Nearest Request:** 2 (distance = $14 - 2 = 12$).
 - **Head Movement:** 14 → 2 (12 cylinders).
6. **Head at 2:**
 - **Pending Requests:** [98, 183, 122, 124]
 - **Nearest Request:** 98 (distance = $98 - 2 = 96$).
 - (Other distances: 122 → 120, 124 → 122, 183 → 181)
 - **Head Movement:** 2 → 98 (96 cylinders).
7. **Head at 98:**
 - **Pending Requests:** [183, 122, 124]
 - **Nearest Request:** 122 (distance = $122 - 98 = 24$).
 - **Head Movement:** 98 → 122 (24 cylinders).
8. **Head at 122:**
 - **Pending Requests:** [183, 124]
 - **Nearest Request:** 124 (distance = $124 - 122 = 2$).

- **Head Movement: 122 -> 124** (2 cylinders).
9. **Head at 124:**
- **Pending Request: 183** (only remaining).
 - **Head Movement: 124 -> 183** (59 cylinders).

11.14.3.3 3.3 Total Head Movement Calculation

Movement	Distance (Cylinders)
53 -> 65	12
65 -> 67	2
67 -> 37	30
37 -> 14	23
14 -> 2	12
2 -> 98	96
98 -> 122	24
122 -> 124	2
124 -> 183	59
Total	236 cylinders

11.14.4 4. Performance Analysis of SSTF

11.14.4.1 4.1 Advantages

- **Reduced Seek Time:** Compared to **FCFS (First-Come, First-Served)**, SSTF significantly **minimizes total head movement**.
 - Example: In FCFS, the head might move arbitrarily far, leading to higher seek times.
- **Efficiency:** Better **average response time** for most requests.

11.14.4.2 4.2 Disadvantages

- **Starvation Problem:**
 - **Definition:** Some requests (especially those far from the initial head position) may be **indefinitely delayed** if new requests keep arriving near the current head position.
 - **Example:** If requests near **cylinder 50-100** keep arriving, a request at **cylinder 183** may never be serviced.
- **Unfairness:** Does not guarantee **fairness** in request handling.

11.14.4.3 4.3 Comparison with FCFS

Metric	SSTF	FCFS
Seek Time	Minimized (optimized)	Potentially high (unoptimized)
Starvation	Possible (distant requests)	No (all requests serviced in order)
Fairness	Low (prioritizes proximity)	High (strict order)
Average Wait Time	Lower	Higher

11.14.5 5. Summary of Key Points

- **SSTF Definition:** A disk scheduling algorithm that services the **nearest request first** to minimize seek time.
- **Operation:**

- Dynamically selects the closest pending request.
- Recalculates distances after each movement.
- **Example Total Head Movement: 236 cylinders** for the given request queue.
- **Performance:**
 - **Pros:** Lower seek time, better efficiency than FCFS.
 - **Cons:** **Starvation risk** for distant requests, lacks fairness.
- **Improvement Suggestions:**
 - **Aging:** Gradually increase priority for long-waiting requests.
 - **Hybrid Approaches:** Combine SSTF with other algorithms (e.g., SCAN, C-SCAN) to mitigate starvation.

11.14.6 6. Conclusion

- SSTF is an **effective** disk scheduling algorithm for **reducing seek time** but suffers from **starvation** issues.
- Further optimizations (e.g., **reordering requests strategically**) can enhance its performance and fairness.

11.15 Stable Storage Implementation

11.15.1 1. Introduction to Stable Storage

11.15.1.1 1.1 Definition of Stable Storage

- **Stable storage** refers to a storage system where **information is never lost**, even in the presence of hardware failures (e.g., disk errors, CPU malfunctions).
- Ensures **data integrity** by guaranteeing that stored data remains **intact and recoverable** despite failures.
- Critical for systems where **data reliability** is paramount (e.g., databases, file systems, transaction logs).

11.15.1.2 1.2 Importance of Stable Storage

- Prevents **data loss** in case of **partial or complete system failures**.
- Ensures **consistency** during recovery, even if additional failures occur mid-recovery.
- Essential for **fault-tolerant systems** where data must persist despite hardware issues.

11.15.2 2. Implementation of Stable Storage

11.15.2.1 2.1 Core Principle: Redundancy

- **Replication** of data across **multiple independent storage devices** with **independent failure modes**.
 - If one device fails, others retain the data.
- **Coordinated updates** ensure that all copies are **consistently updated** to prevent data loss.

11.15.2.2 2.2 Requirements for Stable Storage

1. **Multiple Physical Copies** - At least **two physical blocks** per **logical block** of data.
2. **Atomic Write Operations** - Updates must be **all-or-nothing** (either all copies succeed or none do).
3. **Recovery Mechanism** - Must restore **consistent state** even if failures occur during recovery.

11.15.3 3. Types of Failures in Storage Systems

Three primary failure scenarios affect stable storage:

Failure Type	Description	Impact on Data
Successful Completion	Data is fully and correctly written to disk without errors.	No data loss; system remains consistent.
Partial Failure	Failure occurs mid-write , corrupting some sectors but not all.	Data may be partially corrupted .
Total Failure	Failure occurs before write begins ; disk retains previous data .	No new data is written; old data persists.

11.15.4 4. Ensuring Data Integrity During Writes

11.15.4.1 4.1 Write Operation Procedure To guarantee stable storage, the system follows a **two-phase write process**:

1. **Write to First Physical Block**
 - Data is written to **Block A**.
 - System verifies **successful completion** before proceeding.
2. **Write to Second Physical Block**
 - **Same data** is written to **Block B**.
 - Operation is **only declared complete** if **both writes succeed**.
3. **Completion Condition**
 - If **both writes succeed**, the operation is **atomic and stable**.
 - If **any write fails**, the system **aborts and initiates recovery**.

11.15.4.2 4.2 Why Two Copies Are Sufficient

- **Probability of simultaneous failure** in both blocks is **extremely low** (assuming independent failure modes).
- **More copies** reduce failure risk further but increase **storage overhead**.
- **Two copies** provide a **practical balance** between **reliability** and **cost**.

11.15.5 5. Recovery Process from Failures

11.15.5.1 5.1 Failure Detection & Recovery Steps When a failure is detected, the system examines **both physical blocks** and applies the following rules:

1. **Both Blocks Identical & No Errors**
 - **Action:** No recovery needed; data is **consistent**.
2. **One Block Has Detectable Errors**
 - **Action:** Replace the **corrupt block** with the **intact block's data**.
3. **Blocks Differ but No Detectable Errors**
 - **Assumption:** The **second block (Block B)** contains the **correct data** (since it was written last).
 - **Action:** Overwrite **Block A** with **Block B's content**.

11.15.5.2 5.2 Guarantees of the Recovery Process

- Ensures that **either**:
 - The **write operation fully succeeds** (both blocks updated), **or**
 - **No change occurs** (system reverts to the last consistent state).
- **No intermediate corrupted state** persists after recovery.

11.15.5.3 5.3 Extending to Multiple Copies

- If **more than two copies** are used (e.g., **three blocks**), the recovery process:
 - Uses **majority voting** to determine the correct data.
 - Replaces **corrupt/minority copies** with the **consensus value**.
- **Trade-off:** Higher reliability but **increased storage and write overhead**.

11.15.6 6. Performance Optimization: Non-Volatile RAM (NVRAM)

11.15.6.1 6.1 Role of NVRAM in Stable Storage

- **NVRAM (Non-Volatile RAM)** is used as a **cache** between **volatile memory (RAM)** and **disk storage**.
- **Advantages:**
 - **Faster writes** compared to direct disk writes (since NVRAM has **lower latency**).
 - **Persistent storage** (unlike RAM, NVRAM **retains data after power loss**).
 - **Reduces disk I/O bottlenecks** by batching writes.

11.15.6.2 6.2 How NVRAM Enhances Stable Storage

1. **Data is first written to NVRAM** (fast, non-volatile).
2. **Asynchronously flushed to disk** (improves performance without sacrificing reliability).
3. **If a failure occurs before disk write:**
 - Data is **still safe in NVRAM** and can be **recovered later**.
4. **If NVRAM fails:**
 - Disk copies (from previous stable writes) **ensure data survival**.

11.15.6.3 6.3 Performance Impact

- **Write operations appear faster** to applications (since NVRAM acknowledges completion quickly).
- **Disk writes happen in the background**, reducing **latency spikes**.

11.15.7 7. Summary of Key Concepts

11.15.7.1 7.1 Stable Storage Fundamentals

- **Definition:** Storage where **data is never lost**, even during failures.
- **Implementation:** Achieved via **replication** and **coordinated updates**.

11.15.7.2 7.2 Failure Handling

- **Three failure types:** Successful completion, partial failure, total failure.
- **Recovery mechanism:** Uses **two (or more) copies** to restore consistency.

11.15.7.3 7.3 Write & Recovery Process

- **Atomic writes:** Both copies must succeed for completion.
- **Recovery rules:**
 - If blocks match -> no action.
 - If one is corrupt -> replace with the good copy.
 - If they differ -> **second block is authoritative**.

11.15.7.4 7.4 Performance Optimization

- **NVRAM caching** speeds up writes while maintaining stability.
- **Trade-offs:** More copies improve reliability but increase cost.

11.15.7.5 7.5 Practical Considerations

- **Two copies** are typically **sufficient** for most stable storage implementations.
- **NVRAM** is widely used in **enterprise storage systems** (e.g., databases, RAID controllers).

11.15.8 8. Conclusion

- Stable storage is **essential for fault-tolerant systems** where **data integrity is critical**.
- **Replication + coordinated writes + recovery mechanisms** ensure **no data loss**.
- **NVRAM enhances performance** without compromising reliability.
- Understanding these principles is **key to designing robust storage systems**.

11.16 Swap Space Management

11.16.1 1. Introduction to Swap Space Management

11.16.1.1 1.1 Definition and Purpose

- **Swap space** is a **memory management technique** used in **multiprogramming** environments to allow more processes to share the CPU.
- It involves:
 - Moving a **process out of main memory (RAM)** and storing it in **secondary memory (disk)**.
 - **Bringing it back to main memory** when the process is needed again.
- **Primary goal:** Optimize **memory usage** and improve **system performance** by extending the effective memory capacity.

11.16.1.2 1.2 Role in Virtual Memory Systems

- Acts as an **extension of main memory in virtual memory systems**.
- Although **disk access is significantly slower** than **RAM access**, swap space management aims to:
 - **Maximize throughput** for virtual memory operations.
 - **Minimize performance degradation** due to slow disk I/O.

11.16.2 2. Usage of Swap Space in Operating Systems

11.16.2.1 2.1 Swapping vs. Paging Systems

System Type	Usage of Swap Space
Swapping Systems	Stores entire processes (including code, data, and stack segments).
Paging Systems	Stores individual pages that have been pushed out of main memory .

11.16.2.2 2.2 Factors Influencing Swap Space Requirements

- **Physical memory (RAM) size**
- **Virtual memory usage patterns**
- **System workload (number of concurrent processes)**
- **Swap space requirements** can range from:

- **A few megabytes (MB)** (for lightweight systems)
- **Several gigabytes (GB)** (for memory-intensive applications)

11.16.2.3 2.3 Best Practices for Swap Space Allocation

- **Overestimation is safer** than underestimation:
 - **Underestimating** swap space can lead to:
 - * **Process abortion** (OS kills processes to free memory).
 - * **System crashes** (if no swap space is left).
 - **Overestimating** may waste disk space but **does not harm system functionality**.
- **Historical vs. Modern Approaches:**
 - **Historical Linux recommendation:** Swap space = $2 \times$ **physical RAM**.
 - **Modern Linux systems:** Use **less swap space** due to improved memory management.
 - **Solaris approach:** Swap space = **Virtual memory** – **Pageable physical memory**.

11.16.3 3. Locations of Swap Space

Swap space can reside in **two primary locations**:

11.16.3.1 3.1 File System-Based Swap Space

- **Implementation:**
 - Created as a **large file** within the **existing file system**.
- **Advantages:**
 - **Easy to set up** (no need for dedicated partitioning).
 - **Flexible resizing** (can be adjusted like a regular file).
- **Disadvantages:**
 - **Inefficient performance:**
 - * Requires **navigating directory structures** (slow for large files).
 - * **External fragmentation** can degrade swapping speed.
 - **Higher overhead** due to file system metadata management.

11.16.3.2 3.2 Raw Partition-Based Swap Space

- **Implementation:**
 - Uses a **dedicated disk partition** (not part of the file system).
 - Managed by a **swap space manager** (optimized for speed).
- **Advantages:**
 - **Faster access** (bypasses file system overhead).
 - **Reduced fragmentation** (internal fragmentation is **acceptable**).
 - **Fixed allocation** (swap space is reserved during disk partitioning).
- **Disadvantages:**
 - **Less flexible** (resizing requires repartitioning).
 - **Storage inefficiency** (due to **internal fragmentation**).

11.16.3.3 3.3 Hybrid Approach (Linux Example)

- **Supports multiple swap areas:**
 - Can use **both raw partitions and swap files**.
 - **Flexibility:** Spreads I/O load across multiple storage devices.
 - **Efficiency:** Balances performance and manageability.

11.16.4 4. Comparison: File System vs. Raw Partition Swap Space

Feature	File System Swap Space	Raw Partition Swap Space
Implementation	Large file within the file system.	Dedicated disk partition.
Performance	Slower (file system overhead).	Faster (direct disk access).
Fragmentation	External fragmentation slows swapping.	Internal fragmentation (acceptable).
Flexibility	Easy to resize.	Requires repartitioning.
Management	Higher (file system metadata).	Lower (managed by swap space manager).
Overhead		
Use Case	Suitable for systems with dynamic swap needs.	Preferred for high-performance systems.

11.16.5 5. Swap Space Management in Different Operating Systems

11.16.5.1 5.1 Traditional UNIX

- **Initial Approach:**
 - Copied entire processes between disk and main memory.
- **Evolution with Paging Hardware:**
 - Adopted a **hybrid model** (swapping + paging) for efficiency.

11.16.5.2 5.2 Solaris

- **Improvements:**
 - Optimized swap space usage for **better performance**.
 - Dynamically adjusts swap space based on **virtual memory demands**.

11.16.5.3 5.3 Linux

- **Key Features:**
 - Uses swap space for **anonymous memory** (memory not backed by any file).
 - Supports **multiple swap areas** (files or partitions).
 - **Swap space structure:**
 - * Divided into **4-KB page slots** (each holds a swapped-out page).
 - * **Swap-map:** An **array of integer counters** tracking page usage.
 - **Counter value** = Number of **processes mapping** to the swapped page.
 - **Example:** A counter value of **3** means the page is mapped by **three different processes**.

11.16.6 6. Summary and Key Takeaways

11.16.6.1 6.1 Importance of Swap Space Management

- **Critical for system stability and performance.**
- **Prevents crashes** by ensuring sufficient memory extension.
- **Optimizes virtual memory operations.**

11.16.6.2 6.2 Best Practices

- **Allocate sufficient swap space** (overestimate if unsure).
- **Use raw partitions for performance-critical systems.**
- **Leverage hybrid approaches** (e.g., Linux's multiple swap areas) for **flexibility**.
- **Monitor swap usage** to avoid **performance bottlenecks**.

11.16.6.3 6.3 Final Considerations

- **Swap space is not a substitute for RAM** but an **essential supplement**.
- **Modern OS optimizations** (e.g., Linux's reduced swap reliance) improve efficiency.
- **Understanding swap mechanics** helps in **system tuning** for better performance.